

Java 24 — Generational Shenandoah

Java Version: 24

Year: 2025

Feature: Generational Shenandoah

JEP: 404 — *Generational Shenandoah (Experimental)*

UniqueClassName: GenerationalShenandoah

Generational Shenandoah adds **young and old generations** to Shenandoah GC. The key idea is to collect newly allocated objects more frequently while collecting long-lived objects less frequently. This exploits the common **generational hypothesis**: most newly created objects die young. Java 24 shipped JEP 404 as an **experimental** feature. ([OpenJDK Mail](#))

Advantages and disadvantages

Aspect	Advantages	Disadvantages
GC performance	Focuses collection effort on young objects where there is usually more garbage.	Generational bookkeeping and barriers add implementation complexity.
Allocation throughput	Can reduce the amount of work spent repeatedly examining long-lived objects.	Workloads with unusual object lifetimes may see little benefit or even regression.
Pause times	Retains Shenandoah's concurrent evacuation/compaction approach, helping maintain very short pauses.	Low pause time does not mean zero pauses; application still experiences short GC synchronization points.
Large heaps	Well suited to large heaps because much GC work happens concurrently. Shenandoah's concurrent compaction makes pause behavior less dependent on heap size. (OpenJDK Wiki)	Requires sufficient CPU resources for concurrent GC work.
Memory efficiency	Dead young objects can be reclaimed quickly, potentially reducing memory footprint.	Additional generational metadata and remembered-set/barrier work consume resources.
Old-generation collection	Long-lived objects are not repeatedly processed as aggressively as short-lived objects.	Old-generation reclamation can become important for applications with very high long-lived-object growth.

Object promotion	Objects that survive multiple young collections can eventually move into the old generation. (OpenJDK Mail)	Promotion behavior can make GC tuning more complicated than a simple single-generation collector.
Concurrency	Young-generation collections and evacuations retain Shenandoah's concurrent nature, unlike traditional stop-the-world young-generation collection. (OpenJDK Mail)	Concurrent GC consumes CPU while application threads are running.
Automatic sizing	The implementation can automatically adjust young/old generation capacity to use the heap efficiently. (OpenJDK Mail)	Less direct manual control can make highly specialized tuning more difficult.
Production maturity	It was included in the Java 24 GA release, making it available for experimentation. (OpenJDK Mail)	In Java 24 it was experimental , so production adoption required more caution.
Operational complexity	Can provide better throughput while preserving low-pause characteristics.	GC logs, monitoring, and performance characteristics differ from non-generational Shenandoah.
Workload sensitivity	Excellent when the application creates many short-lived objects.	Less compelling for workloads where most objects are long-lived or allocation rates are low.

When to use Generational Shenandoah

Situation	Use it?	Why	Real-time example
Large heap + low-latency requirement	✓ Yes	Shenandoah is designed for low-pause GC and concurrent	Large-scale trading or market-data processing service

		compaction.	
High object allocation rate	✓ Yes	Young objects can be collected frequently instead of repeatedly processing the entire heap.	REST API receiving thousands of requests/sec
Many short-lived objects	✓ Yes	Excellent match for generational GC.	JSON parsing, DTO creation, HTTP request/response objects
Microservices	✓ Often	Request processing commonly creates many temporary objects.	Spring Boot order/payment service
Real-time-ish applications	✓ Yes	Short and predictable GC pauses can be valuable.	Multiplayer game backend
Large in-memory caches	✓ Potentially	Cache entries tend to become long-lived while temporary application objects remain young.	Product/catalog cache
Event-stream processing	✓ Potentially	Event objects and intermediate transformations are frequently short-lived.	Kafka-based event-processing application
High-throughput web applications	✓ Yes, benchmark first	Frequent temporary allocation makes generational collection attractive.	E-commerce checkout API
Very large Java heap	✓ Potentially	Concurrent collection and compaction can be advantageous.	64–256 GB+ JVM application
Latency-sensitive application	✓ Potentially	Shenandoah targets low pause times.	Financial market-data application

Mostly long-lived objects	☐ Benchmark	Generational advantages become smaller when objects survive for a long time.	Large static object graph
Low allocation application	☐ Usually unnecessary	The additional generational machinery may provide little benefit.	Small batch utility
Small JVM heap	☐ Usually consider alternatives	Shenandoah's strengths may not justify its overhead.	Small command-line Java application
Java 24 production system requiring maximum maturity	☐ Caution	JEP 404 was experimental in Java 24.	Mission-critical banking system
Strictly reproducible GC behavior across versions	☐ Benchmark	GC implementation and heuristics evolve.	Regulated production environment

When not to use it

Situation	Why avoid Generational Shenandoah?	Better consideration
Small heap / simple application	Shenandoah may be unnecessary overhead.	G1 GC
Batch application where pauses don't matter	Low-pause GC may not be the primary requirement.	G1 or Parallel GC
CPU-constrained server	Concurrent GC requires CPU resources.	G1 / Parallel GC depending on workload
Application has mostly long-lived objects	Generational collection provides less advantage.	Benchmark G1, ZGC, or Shenandoah
Very latency-sensitive application but you	GC selection should be based on actual	Benchmark Shenandoah vs ZGC

haven't benchmarked it	pause/allocation measurements.	
Java 24 system avoiding experimental features	JEP 404 was explicitly experimental in Java 24. (OpenJDK Mail)	Use a non-experimental GC
Workload already performs exceptionally well with G1	Switching collectors introduces tuning and operational risk without guaranteed benefit.	Keep G1 if measurements support it
GC behavior is tightly coupled to existing monitoring tooling	Changing collector/mode can change logs and management data.	Keep current collector or validate tooling first

Real-time examples

Application	Object behavior	Why Generational Shenandoah fits
E-commerce API	Every HTTP request creates temporary DTOs, JSON objects, collections, etc.	Most request objects die quickly → young generation is highly useful.
Payment service	Large request volume with temporary transaction-processing objects.	Frequent young collections can reclaim temporary objects efficiently while preserving low pauses.
Trading platform	Huge allocation rate + strict latency requirements.	Concurrent Shenandoah collection can reduce long GC interruptions.
Kafka event processor	Each event creates temporary transformation objects.	Short-lived event objects are natural young-generation candidates.
Search service	Queries create temporary lists, scoring objects, filters, and intermediate results.	Young-generation collection can efficiently reclaim query-specific objects.
Gaming backend	Each game tick creates transient objects while	Separating short-lived objects from long-lived state matches the

	player/session state remains longer-lived.	workload.
Large cache service	Cache data survives for long periods while request-processing objects are temporary.	Long-lived cache objects remain in old generation while temporary objects are collected frequently.
Microservice platform	Many small allocations per request.	Generational behavior can reduce unnecessary old-object processing.

Simple conceptual example

Consider a web service:

```
public OrderResponse createOrder(OrderRequest request) {
    Order order = new Order(request);

    PaymentRequest payment = new PaymentRequest(order);
    PaymentResult result = paymentService.process(payment);

    AuditRecord audit = new AuditRecord(order, result);

    return new OrderResponse(order, result);
}
```

During one request, objects such as:

- OrderRequest
- PaymentRequest
- PaymentResult
- AuditRecord
- temporary collections
- JSON parsing objects
- temporary strings

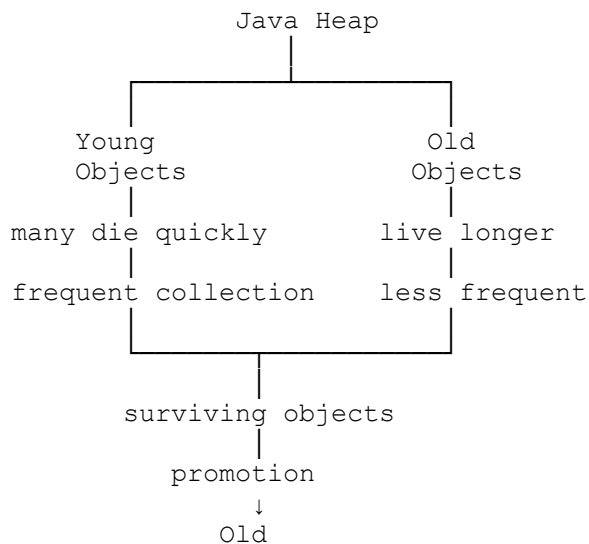
may become unreachable quickly.

Meanwhile:

- Customer
- Product
- Configuration
- Cache entries
- Session information

may remain alive for a much longer time.

Generational Shenandoah tries to exploit this distinction:



The important distinction from traditional generational collectors is that **Shenandoah retains its concurrent evacuation/compaction approach**. Young-generation collections and evacuations can run concurrently, rather than behaving like a conventional stop-the-world young-generation collector. ([OpenJDK Mail](#))

Alternate features

Requirement	Generational Shenandoah	Alternative	When alternative may be better
Low latency + large heap	★★★★★	ZGC	Excellent alternative when extremely low pauses are the primary objective.
General-purpose server GC	★★★★	G1 GC	Strong default choice for many Java applications.
Throughput-oriented application	★★★	Parallel GC	Better when maximum throughput matters more than pause latency.
Low-pause GC	★★★★★	Non-generational Shenandoah	Useful when Shenandoah is desired but generational behavior

			isn't appropriate.
Large heap + low latency	★ ★ ★ ★ ★	ZGC	Particularly attractive for latency-sensitive large heaps.
Traditional generational model	★ ★ ★ ★ ★	G1	Mature, widely used, and automatically manages young/old regions.
Small/simple application	★ ★	G1	Usually simpler and more appropriate.
CPU-constrained workload	★ ★ ★	G1 / Parallel	Concurrent Shenandoah work can consume additional CPU.

Most important alternative: ZGC

For a **large, latency-sensitive application**, the strongest alternative is often **ZGC**.

A simplified choice is:

Requirement	Preferred starting point
Very low latency + large heap	ZGC
Low latency + generational allocation pattern	Generational Shenandoah
General-purpose server application	G1
Maximum throughput, pauses less important	Parallel GC
Want Shenandoah but not generational mode	Shenandoah non-generational mode

One important current-context point: **Java 24's Generational Shenandoah was experimental (JEP 404)**, whereas subsequent JDK work has continued evolving the feature. OpenJDK's later work describes generational Shenandoah as collecting young objects more frequently and old objects less frequently, with potential improvements in allocation throughput, concurrent overhead, and memory footprint for many workloads. (openjdk.org)

Bottom line

Generational Shenandoah is most attractive when you have:

large heap + high allocation rate + many short-lived objects + strict latency requirements

For a typical enterprise application, I would benchmark **Generational Shenandoah vs G1 vs ZGC** rather than selecting it solely from theory. The correct choice depends heavily on allocation rate, live-set size, CPU availability, heap size, and your actual pause/throughput targets.

Java 24 — Compact Object Headers

Java Version: 24

Year: 2025

Feature: Compact Object Headers

JEP: 450 — Compact Object Headers

UniqueClassName: CompactObjectHeaders

Compact Object Headers reduce the amount of memory used by the **object header** of ordinary Java objects. In Java 24, this feature was introduced as a **experimental feature**.

The main motivation is simple: Java applications often create **millions or billions of objects**, so even saving a few bytes per object can significantly reduce heap consumption and improve cache locality.

1. Advantages and disadvantages

Aspect	Advantages	Disadvantages
Memory consumption	Can significantly reduce the memory consumed by object headers.	Benefit depends on the application's object population and object layout.
Heap capacity	More objects can fit into the same heap because each object can require less memory.	Does not reduce the memory occupied by the object's actual fields.
Large object populations	Particularly beneficial when an application creates huge numbers of small objects.	Little benefit when objects are large and contain substantial payload data.
Cache locality	Smaller objects can improve CPU-cache utilization because more objects can fit into cache lines.	Actual improvement depends on access patterns and hardware.

GC efficiency	Smaller object footprints can reduce the amount of memory that garbage collectors need to process.	Compact headers do not fundamentally change the GC algorithm.
Application throughput	Lower memory pressure can reduce allocation and GC pressure in object-heavy workloads.	The performance gain is workload-dependent and should be benchmarked.
Memory bandwidth	Less object data can mean less memory traffic when objects are accessed or copied.	Header manipulation itself may have implementation costs.
Object-oriented applications	Excellent fit for applications with many small Java objects.	Less useful for applications dominated by primitive arrays or large objects.
Synchronization metadata	Header information can be encoded more compactly while preserving required object state.	Object-header representation becomes more sophisticated internally.
Compressed references	Works particularly well with other JVM memory-layout optimizations.	It is not simply the same thing as compressed ordinary object pointers (CompressedOops).
JVM compatibility	Applications generally don't need source-code changes.	Java 24's feature was experimental, so production adoption required caution.
Memory footprint	Can lower overall JVM heap requirements for suitable workloads.	The reduction isn't guaranteed for every object type or JVM configuration.
Migration	Usually transparent to application code.	Requires a JVM that supports and enables the feature.
Future scalability	Useful for extremely object-heavy systems.	As an experimental feature in Java 24, behavior and implementation details may evolve.

2. What exactly is being made compact?

Every ordinary Java object has an **object header** maintained by the JVM.

Conceptually:

Traditional object

Object Header - Mark / synchronization - Class information - Other JVM metadata
Instance fields
Padding / alignment

With Compact Object Headers:

Compact object

Compact Object Header ↓ smaller
Instance fields
Padding / alignment

The important point is that **the application doesn't explicitly modify the class.**

For example:

```
class Customer {  
    long id;  
    String name;  
}
```

You don't rewrite this class to use compact headers.

Instead, the JVM uses a more compact internal representation for the object's header.

3. When to use Compact Object Headers

Scenario	Use?	Reason
Millions of small objects	✓ Yes	Header savings accumulate across a huge number of objects.
Billions of objects	✓ Strong candidate	Even a small saving per object can become substantial.

Large microservice with high allocation rate	✓ Consider	Reducing object footprint can reduce memory pressure.
Large in-memory data structures	✓ Consider	Collections often contain enormous numbers of objects.
High-throughput application	✓ Benchmark	Lower memory traffic may improve throughput.
Large heap	✓ Consider	Memory savings become more valuable at scale.
Memory-constrained container	✓ Potentially	Lower object footprint can help stay within container limits.
Applications with many wrapper objects	✓ Good candidate	Integer, Long, DTOs, nodes, entries, etc. can create large object populations.
Graph/tree structures	✓ Good candidate	Millions of small node objects can make headers significant.
Low-latency application	□ Benchmark	Smaller memory footprint may help cache behavior, but latency benefits aren't guaranteed.
Small application	□ Usually unnecessary	The absolute memory saving may be insignificant.
Large primitive arrays	✗ Usually little benefit	The majority of memory is in array elements rather than individual object headers.
Few large objects	✗ Usually little benefit	Header savings are tiny compared with object payload.

4. When not to use it

Situation	Why not?	Alternative approach
Small Java application	Memory savings may be negligible.	Default JVM configuration
Object count is low	There aren't enough headers for savings to	G1/default GC

	matter.	
Objects are very large	Header size represents a tiny fraction of total memory.	Focus on data representation
Application is already memory-efficient	Complexity/risk may outweigh benefits.	Keep current JVM configuration
Java 24 production system avoids experimental features	Compact Object Headers was experimental in Java 24.	Use standard JVM object headers
Strictly conservative JVM deployment	Experimental JVM functionality may not fit operational policies.	Standard HotSpot configuration
Performance hasn't been measured	Theoretical memory savings don't guarantee application-level performance improvement.	Benchmark first

5. Real-time use cases

Real-time use case	Typical objects	Why Compact Object Headers can help
E-commerce platform	Product, OrderItem, CartItem, DTOs	Millions of small objects can make headers a meaningful part of heap usage.
Banking transaction system	Transaction objects, account events, DTOs	High transaction volumes create many short-lived objects.
Social-media backend	User/event/message objects	Large object populations make per-object memory savings valuable.
Graph database	Vertex/node/edge objects	Graphs can contain enormous numbers of small objects.
Compiler	AST nodes, symbols, tokens	Compilers create large numbers of relatively small objects.
IDE	Syntax trees, PSI objects, symbols	Large projects can create millions of small objects.

Search engine	Query nodes, tokens, intermediate structures	Object-heavy query processing can benefit from reduced object footprint.
Messaging system	Message envelopes, metadata, processing objects	High message throughput generates many temporary objects.
Big-data processing	Records, intermediate objects, task metadata	Object-heavy transformations can create substantial header overhead.
Caching system	Cache entries, wrappers, metadata objects	Millions of small entries can make header memory significant.
Game server	Entity/component objects	Many game entities represented as Java objects can create substantial object overhead.
IoT platform	Sensor events, device messages, metadata	Large numbers of small event objects are common.

6. Example — object-heavy application

Suppose an application creates:

```
class OrderItem {
    long productId;
    int quantity;
    double price;
}
```

And the application maintains:

```
List<OrderItem> items;
```

with **tens or hundreds of millions of OrderItem objects**.

The memory consumed isn't only:

```
productId
quantity
price
```

Each individual object also needs JVM-managed metadata.

Conceptually:

Without compact headers

OrderItem #1

Object header
productId
quantity
price

OrderItem #2

Object header
productId
quantity
price

...

OrderItem #100,000,000

Object header
productId
quantity
price

When the header becomes smaller:

Compact headers

OrderItem #1

Compact hdr
productId
quantity
price

OrderItem #2

Compact hdr
productId
quantity
price

...

The saving **per object may be small**, but multiplied by millions or billions of objects, it can become significant.

7. Particularly good candidates

Application characteristic	Suitability
1 million objects	☐ Moderate
10 million objects	☐ Good
100 million objects	☐ Very good candidate
1 billion+ objects	☐ Excellent candidate
Mostly small objects	☐ Excellent
Mostly large objects	☐ Low
Many object allocations	☐ Excellent
Large primitive arrays	☐ Low
Large byte[] / int[] data	☐ Low
Large graph/tree	☐ Excellent
Huge collection of POJOs/DTOs	☐ Excellent

These are **heuristics, not guaranteed performance thresholds**. Actual benefit depends on object layout, JVM configuration, workload, and hardware.

8. Compact Object Headers vs Compressed Oops

These two concepts are easy to confuse.

Feature	Compact Object Headers	Compressed Oops
Main purpose	Reduce object-header size	Reduce size of object references
Optimizes	Object metadata/header	References/pointers
Main benefit	Smaller individual objects	Smaller references and potentially smaller object layouts

Application code changes	Generally no	Generally no
JVM feature	Yes	Yes
Useful for object-heavy applications	★★★★★	★★★★★
Same feature?	✗ No	✗ No
Can complement each other?	✓ Yes	✓ Yes

Think of it as:

Object memory

Header	← COH
References	← Compressed Oops
Primitive fields	

They address **different parts of object memory**.

9. Alternate features

Requirement	Compact Object Headers	Alternative	Best choice when
Reduce per-object JVM metadata	★★★★★	Compressed Oops	You specifically need smaller references, rather than smaller headers.
Reduce total object count	★★★	Primitive arrays	Data can be represented without one Java object per record.
Reduce object overhead	★★★★★	Off-heap/native data structures	Extremely memory-sensitive systems can use compact external representations.
Improve memory	★★★★★	Records	You need concise

efficiency			immutable data carriers, though records do not inherently eliminate object headers.
Reduce allocation pressure	★ ★ ★ ★	Object reuse/pooling	Objects can safely be reused instead of constantly allocated.
General JVM memory optimization	★ ★ ★ ★ ★	Compressed Class Pointers / Compressed Oops	Standard HotSpot memory optimizations are preferred.
Low-memory data processing	★ ★ ★ ★ ★	Flat/packed data structures	Avoiding millions of individual objects can provide much larger savings than header compression alone.
Standard production configuration	★ ★ ★	Default HotSpot object layout	You want maximum conservatism and don't need experimental features.

10. Compact Object Headers vs other memory optimizations

Technique	What it reduces	Typical benefit
Compact Object Headers	Object-header footprint	☐ Small per-object saving × huge object count
Compressed Oops	Reference size	☐ Smaller object/reference layout
Compressed Class Pointers	Class-pointer representation	☐ Header/class metadata efficiency
Primitive arrays	Object/reference overhead	☐☐☐ Potentially very large
Object pooling	Number of allocations	☐☐ Can reduce allocation/GC overhead
Off-heap storage	Java-heap footprint	☐☐☐ Potentially very large
Data-oriented design	Object count + indirection	☐☐☐ Often extremely effective
Generational GC	GC work on short-lived	Primarily GC performance rather

	objects	than object size
--	---------	------------------

11. Final recommendation

Question	Recommendation
Do I have millions/billions of small objects?	✓ Strong candidate
Is heap memory a major bottleneck?	✓ Investigate
Is object allocation extremely high?	✓ Benchmark
Do I have huge graphs/trees/object collections?	✓ Strong candidate
Do I mainly use large arrays?	✗ Probably little benefit
Do I have only a few large objects?	✗ Probably little benefit
Is Java 24 production stability more important than experimental optimization?	□ Prefer standard configuration
Should I enable it without benchmarking?	✗ No
Is my primary goal reducing GC pauses?	✗ Compact Object Headers isn't primarily a GC feature
Is my primary goal reducing memory footprint?	✓ This is where it is most relevant

Bottom line

Compact Object Headers is primarily a memory-footprint optimization, not a new GC algorithm.

Its sweet spot is:

Huge number of relatively small Java objects → less header overhead → lower heap consumption → potentially better cache/memory behavior.

For a typical enterprise application, the biggest gains usually come from **reducing unnecessary object creation and choosing efficient data representations**; Compact Object Headers becomes

particularly interesting when the application legitimately needs **hundreds of millions or more individual Java objects**.

Java 24 — Prepare to Restrict the Use of JNI

Java Version: 24

Year: 2025

Feature: Prepare to Restrict the Use of JNI

JEP: 472

UniqueClassName: RestrictJNI

Important: This is not a replacement for JNI and does not remove JNI. JEP 472 is a **migration/security preparation feature**. Java 24 begins warning about restricted JNI operations so applications can explicitly declare where native access is required before a future Java release becomes stricter. ([Oracle](#))

The motivation is **integrity by default**: native code can potentially crash the JVM, corrupt memory, bypass Java access checks, or otherwise compromise JVM/application integrity. ([Inside.java](#))

1. Advantages and disadvantages

Aspect	Advantages	Disadvantages
Security	Makes native-code usage more explicit and controlled.	Existing applications using JNI may start producing warnings.
JVM integrity	Helps prevent unrestricted native code from compromising JVM integrity.	JNI itself remains inherently capable of bypassing many Java safety guarantees.
Migration	Gives developers time to prepare for future restrictions.	Requires application/library owners to audit native dependencies.
Visibility	Runtime warnings identify restricted JNI usage and the calling module.	Warning output can become noisy in applications with many native dependencies.
Explicit permissions	--enable-native-access allows native access to be granted selectively.	Deployment scripts and JVM startup configuration may need modification.

Modularity	Native access can be enabled for specific modules rather than the whole application.	Applications on the class path may need ALL-UNNAMED.
Future compatibility	Helps applications become compatible with future Java releases where restrictions become stronger.	Applications ignoring the warnings may eventually fail.
Third-party libraries	Makes hidden native dependencies easier to discover.	You may need to wait for third-party libraries to become compatible.
Operational control	--illegal-native-access lets operators choose allow, warn, or deny behavior in Java 24. (Inside.java)	Incorrect configuration can either hide problems or break applications.
FFM consistency	JNI and FFM native-access behavior becomes more consistent.	Applications using both JNI and FFM may need to review both APIs.
Long-term architecture	Encourages minimizing unnecessary native-code dependencies.	Migrating legacy JNI code can require substantial engineering effort.

2. What does JEP 472 actually restrict?

In Java 24, the following JNI operations are treated as restricted:

JNI operation	Example	Restricted?
System.load()	System.load("/opt/lib/native.so");	✓
System.loadLibrary()	System.loadLibrary("crypto");	✓
Runtime.load()	Runtime.getRuntime().load(path);	✓
Runtime.loadLibrary()	Runtime.getRuntime().loadLibrary("foo");	✓
Declaring a JNI native method	native int calculate();	✓

These restrictions are part of JEP 472's preparation for future native-access restrictions. ([OpenJDK Mail](#))

For example:

```
public class NativeCalculator {  
  
    static {  
        System.loadLibrary("calculator");  
    }  
  
    public native int calculate(int a, int b);  
}
```

Java 24 can issue a warning when restricted JNI operations are performed without the appropriate native-access enablement.

3. How Java 24 handles it

The important options are:

Option	Purpose	Java 24 behavior
--enable-native-access=...	Explicitly grants native access	Recommended long-term mechanism
--illegal-native-access=allow	Allow restricted native access	Permits operation without warning
--illegal-native-access=warn	Warn about illegal native access	Default in Java 24
--illegal-native-access=deny	Reject illegal native access	Throws IllegalCallerException

Oracle documents warn as the Java 24 default, while a future release is intended to make stricter behavior the default. ([Inside.java](#))

Recommended migration configuration

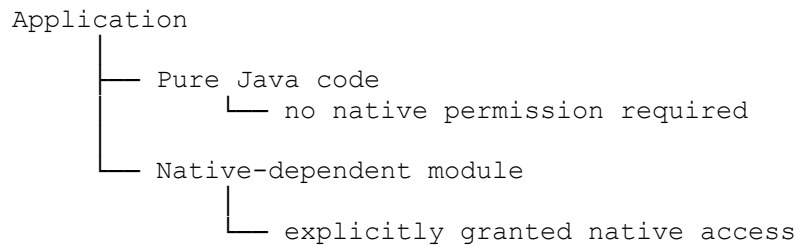
For an application that genuinely requires native access:

```
java \  
  --enable-native-access=com.example.nativeapp \  
  --illegal-native-access=deny \  
  -m com.example.app
```

For an application using the class path rather than named modules:

```
java \
--enable-native-access=ALL-UNNAMED \
--illegal-native-access=deny \
-jar application.jar
```

The idea is:



This is preferable to granting native access indiscriminately.

4. When to use / when not to use

A subtle but important point:

You don't "use" JEP 472 like you use Streams, Records, or Virtual Threads.

Instead, you **use its migration/security mechanism** when your application has native-code dependencies.

Situation	Use / Adopt?	Reason
Application uses JNI	✓ Yes	Explicitly identify and authorize native access.
Application uses <code>System.loadLibrary()</code>	✓ Yes	This is a restricted operation.
Application declares native methods	✓ Yes	JEP 472 specifically addresses this usage.
Application uses native third-party libraries	✓ Yes	Identify and explicitly authorize those dependencies.
Application uses FFM	✓ Yes	JEP 472 aligns native-access restrictions between JNI and FFM.
Application is entirely	☐ Mostly irrelevant	There should be little/no restricted

pure Java		native access to authorize.
Security-sensitive server	✓ Strongly recommended	Explicit native permissions improve control over native-code access.
Modular application	✓ Strongly recommended	Grant native access only to modules that actually need it.
Legacy JNI-heavy application	✓ Migration required	Audit native dependencies before moving to stricter Java releases.
New application needing native functionality	✓ Prefer controlled native access	Avoid unrestricted JNI usage.
Native functionality is available through standard Java APIs	✗ Don't add JNI unnecessarily	Standard Java APIs are generally safer and easier to maintain.
Simple file/network/crypto operation already supported by Java	✗ Usually don't use JNI	Native code adds deployment and integrity risks without sufficient benefit.
Application has no native dependency	✗ No need	There is nothing to authorize.
You are using <code>--illegal-native-access=allow</code> just to suppress warnings	☐ Avoid long-term	This is intended as a short-term compatibility measure. (Inside.java)

5. Real-time use cases

Real-world application	Why JNI is used	JEP 472 impact	Recommendation
Video processing	FFmpeg/native codecs	Native libraries need explicit access	✓ Explicitly enable native access
Machine learning	Native inference libraries	High-performance native libraries may be loaded	✓ Explicitly authorize required modules
Database drivers	Native database client libraries	Driver may load native libraries	✓ Audit driver and authorize appropriately

Cryptography	Hardware/native crypto libraries	Native implementations can provide hardware acceleration	✓ Use only where justified
Image processing	Native image codecs/algorithms	Native libraries may provide performance	✓ Explicit native access
Operating-system integration	Native OS APIs	JNI can access functionality unavailable through Java APIs	✓ Appropriate use case
GPU computing	CUDA/OpenCL/native GPU libraries	Java application communicates with native runtime	✓ Explicitly authorize native module
High-performance networking	Native networking libraries	Native code can provide specialized networking capabilities	□ Benchmark and authorize
Legacy enterprise application	Old JNI libraries	Existing native dependencies may trigger warnings	✓ Audit and migrate
Pure Spring Boot REST API	Usually none	No reason to introduce JNI merely because JEP 472 exists	✗ Stay pure Java
Java desktop application using native OS integration	Native desktop APIs	May need JNI/native libraries	✓ Explicitly authorize
Embedded Java application	Device-specific native APIs	Hardware often requires native integration	✓ Appropriate with strict control

6. Example — legacy JNI application

Suppose an application has:

```

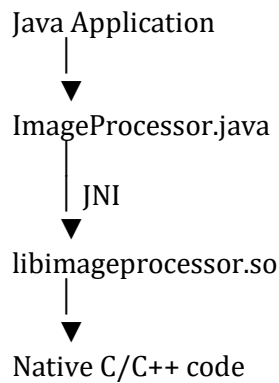
public class ImageProcessor {

    static {
        System.loadLibrary("imageprocessor");
    }

    public native byte[] resize(byte[] image, int width, int height);
}

```

And:



In Java 24, the native loading/declaration is subject to the new restricted-native-access rules.

Instead of ignoring the warning, explicitly configure the application:

```

java \
  --enable-native-access=ALL-UNNAMED \
  --illegal-native-access=deny \
  -jar image-service.jar

```

This communicates:

"This application intentionally requires native access."

That is fundamentally different from allowing arbitrary code on the class path to access native functionality without explicit approval.

7. Example — modular application

Suppose:

```

com.company.application
com.company.payment
com.company.nativecrypto

```

Only:

`com.company.nativecrypto`

needs JNI.

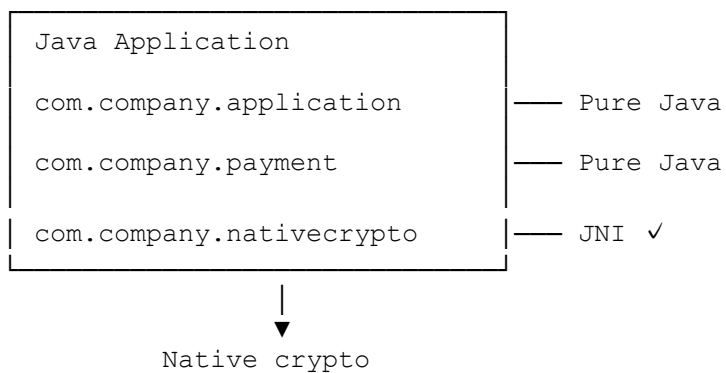
Instead of:

`--enable-native-access=ALL-UNNAMED`

prefer:

`--enable-native-access=com.company.nativecrypto`

Conceptually:



This follows the principle of **least privilege**.

8. Advantages vs disadvantages for enterprise applications

Enterprise concern	JEP 472 effect
Security	☐ Better
JVM integrity	☐ Better
Native dependency visibility	☐ Better
Migration effort	☐ Higher
Legacy compatibility	☐ Requires testing
Deployment configuration	☐ More configuration

Third-party dependency management	☐ Requires auditing
Future Java compatibility	☐ Better
Operational predictability	☐ Better once configured
Application source-code changes	☐ Usually minimal
Build/deployment changes	☐ Potentially required

9. Alternate features / approaches

There isn't one feature that is a **direct replacement** for JEP 472 because JEP 472 is primarily a **security/migration mechanism**, rather than an API for performing native calls.

However, there are several alternatives depending on why you currently use JNI.

Requirement	JNI + JEP 472	Alternative	When alternative is better
Call native C/C++ functions	★ ★ ★ ★	Foreign Function & Memory API (FFM)	New native integrations where FFM provides the required functionality.
Access native memory	★ ★ ★	FFM API	Preferable for modern Java native interoperability.
Native library integration	★ ★ ★ ★	FFM API	Avoids much of the traditional JNI boilerplate.
OS functionality	★ ★ ★ ★	Standard Java API	Prefer Java APIs whenever the required functionality already exists.
Cryptography	★ ★ ★	JCA/JCE	Prefer standard Java cryptographic APIs when sufficient.
Networking	★ ★ ★	Java networking APIs	Avoid native code when Java provides the required

			functionality.
Database access	★ ★ ★	JDBC	Prefer JDBC instead of custom JNI database integration where possible.
High-performance native computation	★ ★ ★ ★ ★	FFM + native library	FFM can provide a modern native interop mechanism.
GPU integration	★ ★ ★ ★ ★	FFM / specialized GPU APIs	Depends on the GPU ecosystem and library.
Legacy native application	★ ★ ★ ★ ★	Keep JNI + explicitly enable native access	Migration to another mechanism may not justify the cost.

10. JNI vs FFM

For **new native integrations**, this is one of the most important comparisons.

Feature	JNI	FFM API
Age	Very mature	Modern API
Native integration	✓	✓
Native memory access	✓	✓
Boilerplate	Higher	Generally lower
Requires native method declarations	Often	No traditional JNI native methods
Java/C boundary	More manual	More Java-level API support
Existing ecosystem	★ ★ ★ ★ ★	★ ★ ★ ★
Legacy compatibility	★ ★ ★ ★ ★	★ ★ ★
New projects	☐ Consider	☐ Often preferable
JEP 472 impact	Restricted native operations	Also subject to native-access restrictions

Security model	Native access must be controlled	Native access must be controlled
Future direction	Still supported	Important modern alternative

JEP 472 **does not mean "replace all JNI with FFM immediately."** It prepares both JNI and FFM for a common native-access security model. ([Oracle](#))

11. Practical migration strategy

Step	Action	Goal
1	Upgrade/test on Java 24	Detect JNI warnings
2	Identify native libraries	Know exactly what uses native code
3	Identify calling modules	Determine where native access originates
4	Use jnativescan where appropriate	Statically discover restricted JNI usage. (Inside.java)
5	Replace unnecessary JNI	Reduce native attack/integrity surface
6	Consider FFM for new integrations	Use modern native interoperability where suitable
7	Configure --enable-native-access	Explicitly authorize legitimate native access
8	Test with --illegal-native-access=deny	Find code that isn't correctly authorized
9	Fix third-party dependencies	Upgrade libraries that aren't ready
10	Keep native access narrowly scoped	Follow least privilege

12. Final comparison

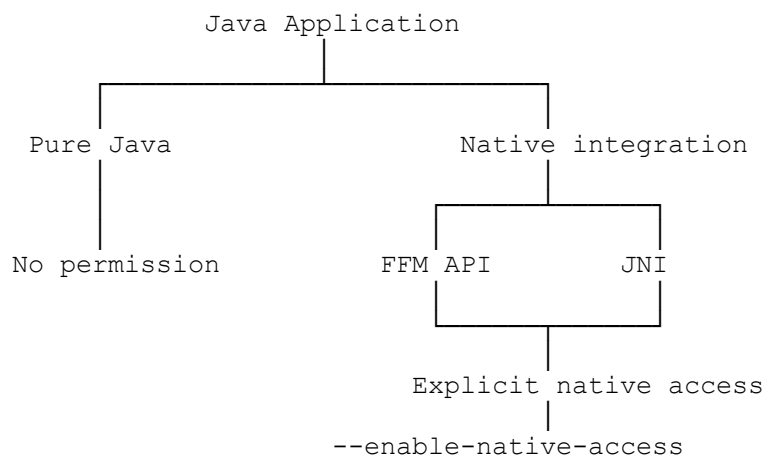
Question	Recommendation
Is JEP 472 a new JNI replacement?	✗ No
Does it remove JNI?	✗ No

Does Java 24 warn about restricted JNI operations?	✓ Yes
Is the purpose mainly security/integrity and migration?	✓ Yes
Should all JNI code be removed?	✗ No
Should legitimate JNI usage be explicitly authorized?	✓ Yes
Should unnecessary JNI be removed?	✓ Yes
Should new native integrations consider FFM?	✓ Yes
Should --illegal-native-access=allow be the permanent solution?	✗ No
Should applications test with --illegal-native-access=deny?	✓ Yes
Should native access be granted to the smallest possible set of modules?	✓ Yes
Is this important for Java-version migration?	★ ★ ★ ★ ★

Bottom line

JEP 472 is essentially a "prepare now, before future Java versions enforce it" feature.

The recommended architecture is:



For **existing JNI-heavy systems**, don't panic or blindly rewrite everything. **Audit → explicitly authorize → test with deny → remove unnecessary JNI → consider FFM for new native integrations.** Java 24's warnings are specifically intended to give applications time to make this transition before future releases become stricter. ([Oracle](#))

Java 24 — Late Barrier Expansion for G1

Java Version: 24

Year: 2025

Feature: Late Barrier Expansion for G1

JEP: 475 — Late Barrier Expansion for G1

UniqueClassName: LateBarrierExpansionG1

Late Barrier Expansion is primarily a **JVM/JIT implementation optimization for G1 GC**. It changes *when* G1's GC barriers are expanded into their lower-level instructions: instead of expanding them relatively early during compilation, expansion is delayed until later in the compilation pipeline.

The goal is mainly to **improve generated-code quality and compilation efficiency**, rather than to change the G1 garbage-collection algorithm itself.

1. Advantages and disadvantages

Aspect	Advantages	Disadvantages
JIT compilation	Delaying barrier expansion gives the compiler more opportunity to optimize surrounding code before barriers become complex instruction sequences.	Makes the compiler pipeline more complicated.
Generated machine code	Can produce better-optimized machine code in some workloads.	Improvement is workload- and code-path-dependent.
G1 GC	Retains G1's existing GC behavior while optimizing how its barriers are compiled.	Does not fundamentally change G1's collection algorithm.
Compilation optimization	Allows optimizations to operate on simpler intermediate representations for longer.	More sophisticated compiler implementation is harder to maintain.
Barrier code	Barrier expansion happens closer to the point where final machine-code generation is performed.	Barrier handling becomes more tightly integrated with late compilation stages.
Runtime performance	Can reduce unnecessary	There is no guarantee of measurable

	instructions and improve generated-code efficiency.	application-level speedup.
CPU efficiency	Better generated code can potentially reduce CPU overhead.	Benefits may be too small to notice for CPU-light applications.
JIT optimization	Gives the compiler additional freedom before committing to platform-specific barrier implementation.	More complex compiler behavior can make debugging compiler issues harder.
Memory access	Potentially improves optimization around memory accesses containing G1 barriers.	Does not reduce the fundamental need for G1 barriers.
GC overhead	May reduce some barrier-related execution overhead.	G1 still performs the same fundamental remembered-set/card-marking/barrier responsibilities.
Application compatibility	Application source code normally requires no changes.	Benefits depend on using G1 and the relevant JVM/JIT implementation.
Deployment	Normally transparent to developers.	There is generally no application-level API to "use" directly.

2. What problem does it solve?

G1 needs **GC barriers** around certain memory operations.

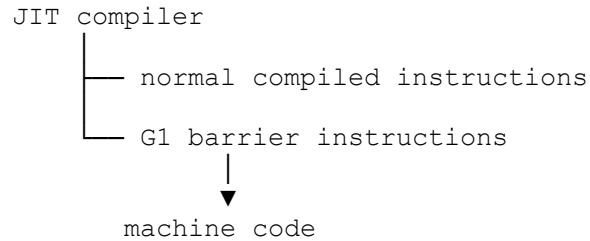
For example, conceptually:

```
object.field = value;
```

The JVM may need additional GC-related work around such an operation.

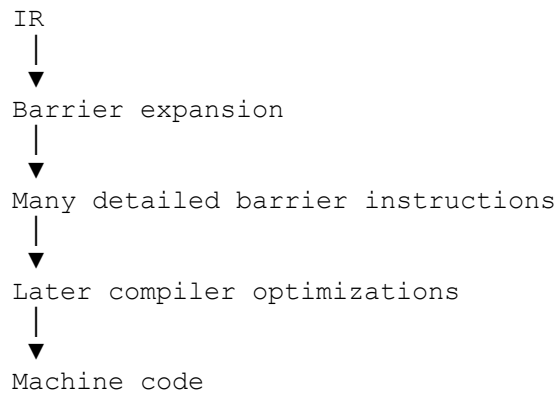
Conceptually:

```
Java source
  |
  ▼
object.field = value
  |
  ▼
```

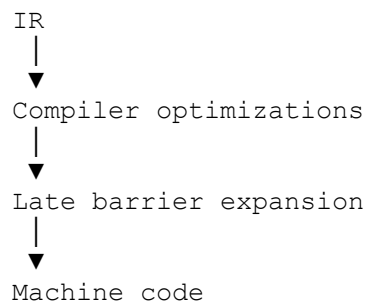


The important change in JEP 475 is approximately:

Before



JEP 475



In other words, the compiler **keeps barriers in a higher-level representation for longer** and expands them later.

This gives the compiler more freedom to optimize the surrounding code before the barrier implementation becomes fully expanded.

3. When to use

Because this is a JVM implementation feature, the question is slightly different from features such as Records or Virtual Threads.

You generally don't write:

```
useLateBarrierExpansion();
```

Instead, you benefit from it when running an appropriate Java 24 JVM using **G1**.

Situation	Use?	Reason
Application uses G1	✓ Yes	This feature specifically improves G1 barrier compilation.
Large enterprise application	✓ Yes	Small runtime/JIT improvements can matter at scale.
Long-running server	✓ Yes	JIT compilation quality matters significantly after code becomes hot.
High-throughput API	✓ Yes	Large numbers of compiled memory operations can make compiler efficiency important.
High object mutation rate	✓ Yes	G1 barriers can occur frequently around relevant memory operations.
Large Java application with many hot methods	✓ Yes	More JIT-compiled code means more opportunity for compiler improvements.
CPU-sensitive workload	✓ Yes	Reduced generated-code overhead can potentially improve CPU efficiency.
Low-latency application	□ Potentially	More efficient compiled code may help, but benchmark actual latency.
Application already using G1 defaults	□ Yes	The improvement is intended to be largely transparent.
Application uses ZGC	✗ No direct benefit	JEP 475 specifically targets G1 barrier expansion.
Application uses Shenandoah	✗ No direct benefit	This JEP is about G1.
Application uses Parallel GC	✗ No direct benefit	Not a G1 barrier optimization.
Short-lived command-	□ Limited	There may not be enough JIT

line program		compilation time to realize much benefit.
Interpreter-heavy application	☐ Limited	The optimization concerns compiled code.

4. When not to use / consider alternatives

Situation	Why not prioritize it?	Better approach
You don't use G1	The feature is specifically related to G1.	Choose optimizations relevant to your collector.
You use ZGC	G1 barrier expansion isn't applicable.	Focus on ZGC tuning and application allocation behavior.
You use Shenandoah	Same reason.	Focus on Shenandoah-specific tuning.
Tiny application	JIT/barrier optimization may have negligible impact.	Keep default JVM configuration.
Application startup time dominates	Late compiler optimizations help less when the application terminates quickly.	Consider startup-oriented JVM choices.
Main bottleneck is database latency	GC barrier optimization won't fix slow database calls.	Optimize SQL, connection pools, indexes, etc.
Main bottleneck is network latency	CPU/JIT optimization is unlikely to be the primary solution.	Optimize networking and I/O.
Application is allocation-bound	Reducing unnecessary allocations can have a much larger effect.	Improve object allocation patterns.
Application has excessive GC pressure	Barrier optimization alone won't solve excessive allocation.	Analyze heap/allocation profiles and GC behavior.
You need a guaranteed	JEP 475 doesn't guarantee	Benchmark the actual workload.

performance improvement	an application-level speedup.	
--------------------------------	-------------------------------	--

5. Real-time use cases

Real-world application	G1 activity	Potential benefit
E-commerce API	Huge number of requests modify objects and collections.	More efficient compiled G1 barrier code may reduce CPU overhead.
Banking transaction service	High transaction throughput produces frequent object mutations.	Potential improvement in compiled memory-operation efficiency.
Large Spring Boot service	Many hot Java methods and high allocation/mutation rates.	Better JIT-generated code can improve throughput in hot paths.
Microservices platform	Thousands of service calls and object transformations.	Small per-operation savings can accumulate.
Order-processing system	Large numbers of object field updates.	G1 barrier handling is frequent enough to make compiler efficiency relevant.
In-memory cache	Many reads/writes and reference updates.	Potentially lower generated barrier overhead.
Search engine	Query execution generates and modifies many objects.	Better machine code may improve CPU efficiency.
Message-processing system	Millions of messages create and mutate temporary objects.	Hot message-processing code can benefit from JIT improvements.
Large Java application server	Long-running hot methods are heavily JIT compiled.	More opportunity for late barrier optimization.
Real-time-ish service	GC and CPU overhead both affect latency.	Potentially better CPU efficiency, subject to benchmarking.

6. Example — high-throughput service

Consider:

```
class Order {  
    Customer customer;  
    Payment payment;  
    String status;  
}
```

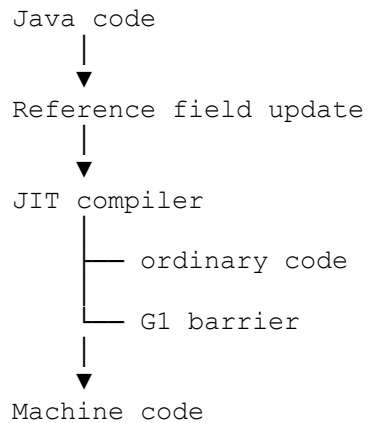
A high-throughput service might repeatedly execute:

```
order.customer = customer;  
order.payment = payment;  
order.status = "PAID";
```

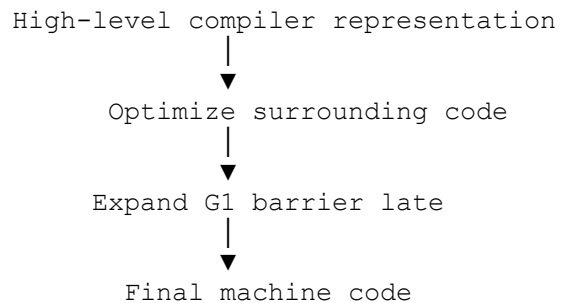
At the Java source level this looks simple.

But the JVM may generate GC-related barrier operations around relevant reference stores.

Conceptually:



With late barrier expansion:



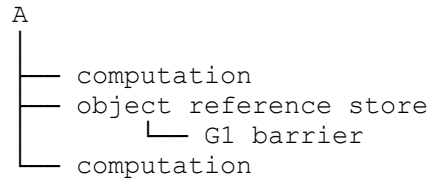
The application developer doesn't have to rewrite:

```
order.customer = customer;
```

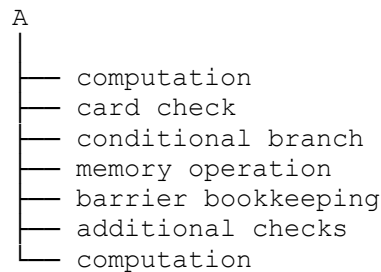
The optimization happens **inside the JVM's compiler pipeline**.

7. Why delaying barrier expansion can help

Suppose the compiler initially sees:

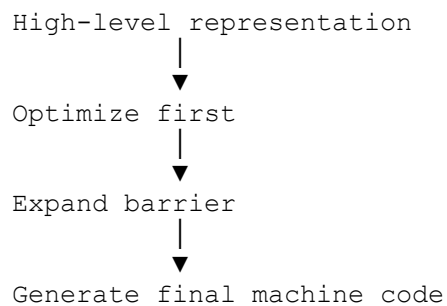


If the barrier is expanded too early, the compiler may see a much more complicated representation:



That complexity can make some compiler optimizations harder.

Late expansion instead allows:



So the core idea is:

Keep G1 barriers abstract for longer, optimize the program around them, and expand them closer to final code generation.

8. Advantages for different application types

Application type	Expected value
High-throughput server	★ ★ ★ ★ ★
Large enterprise application	★ ★ ★ ★
Long-running JVM	★ ★ ★ ★
Object-heavy application	★ ★ ★ ★
CPU-bound application	★ ★ ★ ★
High mutation rate	★ ★ ★ ★
Low-latency G1 application	★ ★ ★ ★
Microservice	★ ★ ★
Small CLI application	★
Short-lived JVM	★
I/O-bound application	★ ★

These are **relative expectations**, not guaranteed benchmark results.

9. Alternate features

There is no direct application-level replacement for JEP 475 because it is an **internal G1/JIT optimization**.

The alternatives depend on the problem you're trying to solve.

Goal	Late Barrier Expansion for G1	Alternative	When alternative is better
Improve G1 performance	★ ★ ★ ★ ★	G1 tuning/application optimization	When GC configuration or allocation behavior is the actual bottleneck.
Reduce GC pauses	★ ★ ★	ZGC	When extremely low pause latency is more important than G1 compatibility.

Reduce GC pauses while using Shenandoah	★ ★ ★	Shenandoah	When Shenandoah's characteristics better match the workload.
Improve throughput	★ ★ ★ ★	Parallel GC	When throughput matters more than pause times.
Reduce G1 allocation pressure	★ ★ ★	Reduce object allocations	Often more impactful than JVM compiler changes.
Reduce object overhead	★ ★ ★	Compact Object Headers	Useful when millions/billions of objects dominate memory.
Improve native interoperability	✗	FFM API	Completely different problem: native library/memory access.
Improve concurrent processing	✗	Virtual Threads	When blocking I/O and thread scalability are the bottleneck.
Improve JIT optimization	★ ★ ★ ★ ★	General JIT optimizations	Relevant when CPU execution rather than GC is the primary bottleneck.

10. G1 vs ZGC vs Shenandoah

If you're actually deciding which JVM/GC technology to use:

Requirement	G1 + Late Barrier Expansion	ZGC	Shenandoah
General-purpose server	★ ★ ★ ★ ★	★ ★ ★ ★	★ ★ ★ ★
Large heap	★ ★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★ ★ ★
Low latency	★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★ ★ ★
Throughput	★ ★ ★ ★ ★	★ ★ ★ ★	★ ★ ★ ★

Mature general-purpose choice	★ ★ ★ ★ ★	★ ★ ★ ★	★ ★ ★ ★
G1 ecosystem	★ ★ ★ ★ ★	✗	✗
Generational capabilities	★ ★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★ ★ ★
JEP 475 benefit	✓	✗	✗
Application code changes	None	Usually none	Usually none

11. How it fits into Java 24 performance improvements

This feature is best understood as part of a **JVM implementation optimization**, rather than an application programming feature.

Java 24 feature	Primary area	Developer-visible?	Main objective
Late Barrier Expansion for G1	JIT + GC integration	✗	Improve G1 barrier compilation
Compact Object Headers	JVM object layout	✗	Reduce object memory footprint
Generational Shenandoah	Garbage collection	□	Improve Shenandoah's generational behavior
Prepare to Restrict JNI	Security/native access	✓	Prepare for stricter native-access rules
Stream Gatherers	Java API	✓	Custom intermediate stream operations
Class-File API	JVM/tooling	✓	Programmatic class-file manipulation

12. Final recommendation

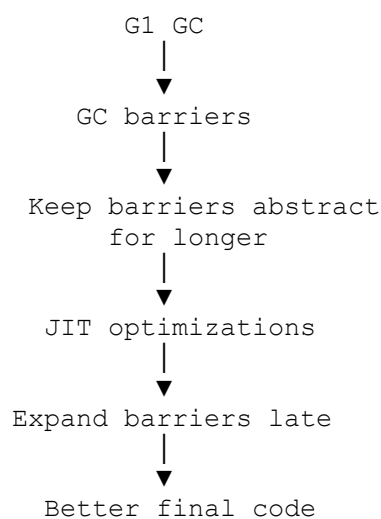
Question	Recommendation
----------	----------------

Do I need to change Java source code?	✗ No
Do I need to call an API?	✗ No
Is this primarily a JVM implementation optimization?	✓ Yes
Does it specifically target G1?	✓ Yes
Does it change the G1 GC algorithm?	✗ No
Can it improve generated machine code?	✓ Potentially
Can it improve CPU efficiency?	✓ Potentially
Will every application become faster?	✗ No
Is it most relevant to long-running G1 applications?	✓ Yes
Is it relevant to ZGC/Shenandoah?	✗ Not directly
Should you tune your application specifically for it?	✗ Generally no
Should you benchmark after upgrading to Java 24?	✓ Yes

Bottom line

Late Barrier Expansion for G1 is a "behind-the-scenes" Java 24 performance improvement.

Its basic idea is:



So, unlike **Compact Object Headers**, which primarily targets **memory footprint**, and **Generational Shenandoah**, which changes GC organization, **Late Barrier Expansion for G1** primarily improves the interaction between G1's barriers and the JIT compiler.

For most developers, the practical advice is simple: **if you're running G1 on Java 24, you automatically benefit from the JVM's implementation; don't write special application code for it.** Measure application throughput, CPU utilization, allocation rate, and latency when evaluating the overall impact.

Java 24 — Key Derivation Function API (Preview)

Java Version: 24

Year: 2025

Feature: Key Derivation Function API (Preview)

JEP: 478

UniqueClassName: KeyDerivationFunctionAPI

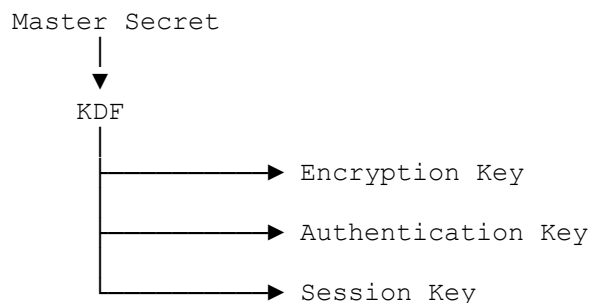
JEP 478 introduces a standard `javax.crypto.KDF` API for **deriving cryptographic key material from existing secret/input keying material (IKM) and optional additional data**. Java 24's API is a **preview feature**, so it must be enabled explicitly and its API was not yet permanent in Java 24. ([OpenJDK Bug System](#))

The initial API is designed to support KDF algorithms such as **HKDF (RFC 5869)** and to provide a provider-based architecture that can accommodate other KDFs. ([OpenJDK Bug System](#))

1. What problem does KDF solve?

A KDF is used when you have some secret key material and need to derive **one or more independent cryptographic keys** from it.

For example:



Instead of manually implementing a construction such as:

```
SHA-256(secret + "encryption")
SHA-256(secret + "authentication")
SHA-256(secret + "session")
```

a standardized KDF can perform key derivation according to a defined cryptographic construction.

The Java 24 API provides `KDF.deriveKey()` for producing a `SecretKey` and `KDF.deriveData()` for producing raw derived data. ([Oracle Docs](#))

2. Advantages and disadvantages

Aspect	Advantages	Disadvantages
Standard API	Provides a standard Java SE API for KDF operations instead of requiring every application/provider to invent its own interface.	Java 24 API was a preview feature and therefore subject to change.
Cryptographic correctness	Encourages use of standardized KDF constructions such as HKDF instead of ad-hoc hashing schemes.	Developers still need to choose correct algorithms and parameters.
HKDF support	Provides APIs specifically for HKDF Extract, Expand, and Extract-then-Expand operations. (Oracle Docs)	Understanding HKDF parameters is still necessary for secure protocol design.
Provider architecture	Security providers can implement KDF algorithms through KDFSpi. (Oracle Docs)	Provider compatibility can vary by algorithm and Java version.
Key separation	Makes it easier to derive independent keys for different purposes from common input key material.	Incorrect context/info parameters can undermine intended key separation.
Protocol development	Useful for modern protocols that need multiple derived secrets.	It does not automatically design the complete protocol for you.

Integration with cryptography	Works naturally with <code>SecretKey</code> and the existing Java Cryptography Architecture.	Developers still need to manage the original secret material securely.
Raw key material	<code>deriveData()</code> can produce raw bytes when a <code>SecretKey</code> isn't appropriate. (Oracle Docs)	Raw byte handling increases the responsibility on application developers.
Extensibility	Provider SPI allows additional KDF implementations.	Additional algorithms may require a third-party or custom security provider.
Performance	Native/provider implementations can potentially optimize KDF operations.	KDF operations can intentionally be computationally expensive depending on the algorithm.
Security maintenance	Centralizes KDF functionality in the Java cryptographic architecture.	Preview APIs should not be treated as permanently stable in Java 24.
Future cryptography	Designed to support KDF usage in modern protocols such as HPKE and KEM-based schemes. (OpenJDK Bug System)	Applications adopting preview APIs may need migration work when the API evolves.

3. Core API

The major Java 24 classes are:

Class	Purpose
<code>javax.crypto.KDF</code>	Main API for performing key derivation
<code>javax.crypto.KDFSpi</code>	Service Provider Interface for implementing KDF algorithms
<code>javax.crypto.KDFParameters</code>	Base abstraction for KDF parameters
<code>javax.crypto.spec.HKDFParameterSpec</code>	HKDF-specific parameters
<code>HKDFParameterSpec.Extract</code>	Parameters for HKDF Extract

HKDFParameterSpec.Expand	Parameters for HKDF Expand
HKDFParameterSpec.ExtractThenExpand	Parameters for the combined operation

These classes are part of Java 24's preview API. ([Oracle Docs](#))

4. Basic example

A conceptual HKDF usage looks like:

```
import javax.crypto.KDF;
import javax.crypto.SecretKey;
import javax.crypto.spec.HKDFParameterSpec;

byte[] inputKeyMaterial = ...;

KDF kdf = KDF.getInstance("HKDF-SHA256");

HKDFParameterSpec params =
    HKDFParameterSpec.ofExtract()
        .addIKM(inputKeyMaterial)
        .build();

SecretKey key = kdf.deriveKey("AES", params);
```

The exact API usage depends on the selected HKDF operation and parameters. The Java 24 API provides builder-style specifications for Extract, Expand, and Extract-then-Expand. ([Oracle Docs](#))

Because this is a preview API in Java 24, compilation and execution require preview support, for example:

```
javac --release 24 --enable-preview Main.java
java --enable-preview Main
```

([OpenJDK Bug System](#))

5. When to use KDF API

Situation	Use?	Why
Deriving multiple keys from one master secret	✓ Yes	KDFs are specifically designed for this.

Session-key derivation	✓ Yes	Common cryptographic protocol requirement.
Key separation	✓ Yes	Different derived keys can be associated with different purposes.
TLS-like key schedules	✓ Yes	Modern protocols commonly use structured key derivation.
HKDF-based protocol implementation	✓ Yes	Java provides a standard HKDF API.
Implementing HPKE-like protocols	✓ Yes	KDFs are an important component of HPKE key schedules.
KEM-based cryptography	✓ Yes	KDFs are commonly used after shared-secret establishment.
Deriving encryption and authentication keys	✓ Yes	Helps establish cryptographic key separation.
Secure token/session infrastructure	□ Often	Useful when a protocol requires deterministic derivation of key material.
Building a cryptographic library/provider	✓ Yes	KDFSpi provides the provider extension mechanism.
Password hashing	□ Not automatically	Password hashing needs an appropriate password-based KDF such as Argon2/scrypt/PBKDF2 and suitable parameters.
Simple hashing	✗ No	Use MessageDigest when you only need a hash.
Digital signatures	✗ No	Use Signature.
Encrypting data directly	✗ No	Use Cipher; KDF derives keys rather than encrypting application data.

6. When not to use it

Situation	Why not use KDF?	Better API/approach
-----------	------------------	---------------------

Need to encrypt plaintext	KDF does not encrypt data.	Cipher
Need to calculate a hash	KDF is unnecessary.	MessageDigest
Need a random AES key	Don't derive it unnecessarily.	KeyGenerator
Need a digital signature	KDF isn't a signature mechanism.	Signature
Need public-key encryption	KDF isn't an encryption mechanism.	Cipher / appropriate KEM
Need password storage	Generic HKDF is not a password-storage solution.	A password hashing/KDF scheme designed for passwords
Existing protocol already specifies a different KDF	Changing the algorithm may break protocol interoperability.	Follow the protocol specification
Simple application with no key-derivation requirement	Adds unnecessary cryptographic complexity.	Existing Java security APIs
Production application that cannot accept preview APIs	Java 24 KDF API is preview.	Use a stable provider/library API or wait for a finalized JDK API

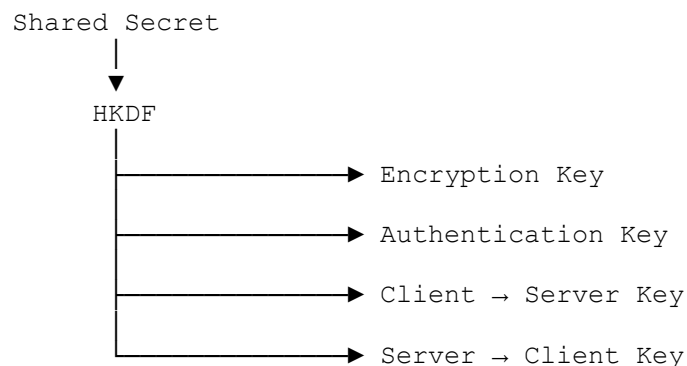
7. Real-time use cases

Real-time use case	Input	Derived output	Why KDF is useful
TLS-style session establishment	Shared secret	Multiple traffic keys	Separates encryption/authentication/key-direction material.
Secure messaging	Shared conversation secret	Message/session keys	Avoids using one master secret directly for every operation.
VPN session	Handshake secret	Encryption keys	Provides session-specific derived keys.
Payment gateway	Master/session secret	Transaction-specific key material	Enables controlled key separation.

Cloud service authentication	Root secret	Service-specific keys	Limits direct reuse of a root key.
API session security	Session secret	Encryption/signing keys	Separates cryptographic purposes.
IoT device communication	Device secret	Per-session/per-purpose keys	Useful for constrained devices and secure channels.
File encryption system	Master key	File-specific keys	Limits direct reuse of the master key.
Database encryption	Master key	Data-encryption keys	Enables hierarchical key-management designs.
Secure messaging protocol	Handshake/shared secret	Sending/receiving keys	Provides directional key separation.
KEM-based protocol	Shared KEM secret	Symmetric session keys	KDF converts shared secret material into usable symmetric keys.
HPKE implementation	Shared secret	Key schedule material	KDF is fundamental to the HPKE key schedule.

8. Example — derive different keys from one secret

Suppose an application establishes:



Instead of using:

```
byte[] sharedSecret = ...;

// Bad conceptual design:
byte[] encryptionKey = sharedSecret;
byte[] authenticationKey = sharedSecret;
```

a protocol can derive separate key material using different context information.

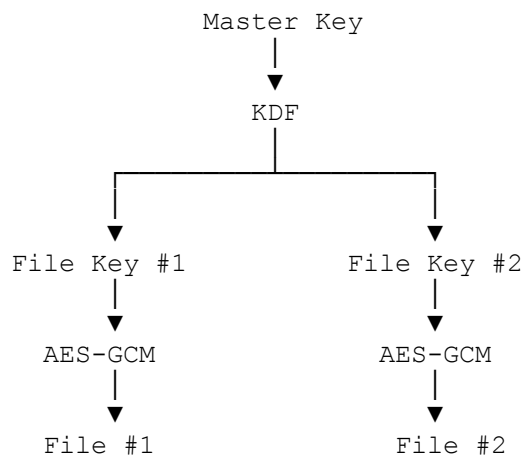
Conceptually:

```
HKDF(sharedSecret, "encryption")
HKDF(sharedSecret, "authentication")
HKDF(sharedSecret, "client-to-server")
HKDF(sharedSecret, "server-to-client")
```

The actual protocol should follow the relevant cryptographic specification rather than inventing labels arbitrarily.

9. Example — secure file encryption architecture

A useful architecture might be:



The advantage is **key separation**.

Compromise or replacement of one derived key does not require treating every other purpose as using the same key.

10. KDF vs common Java cryptographic APIs

API	Purpose	KDF API replacement?
MessageDigest	Hash data	✗
Mac	Message authentication code	✗
Cipher	Encryption/decryption	✗
Signature	Digital signatures	✗
KeyGenerator	Generate random symmetric keys	✗
KeyAgreement	Establish shared secret	✗
KDF	Derive new key material from existing secret material	✓
SecureRandom	Generate cryptographically secure random values	✗
KEM APIs	Establish encapsulated shared secrets	✗ but KDF may be used afterward

The important distinction is:

KeyAgreement / KEM



Shared Secret



KDF



Usable application/session keys

11. Advantages for enterprise applications

Enterprise requirement	KDF API value
Standard Java cryptographic API	★ ★ ★ ★ ★
Key separation	★ ★ ★ ★ ★

Session-key generation	★ ★ ★ ★ ★
Protocol implementation	★ ★ ★ ★ ★
Provider integration	★ ★ ★ ★ ★
HKDF support	★ ★ ★ ★ ★
Integration with JCA	★ ★ ★ ★ ★
Reduced custom crypto code	★ ★ ★ ★
Legacy compatibility	★ ★ ★
Stable Java 24 API	★ ★ — preview
Easy cryptographic design	★ ★ ★
Performance	★ ★ ★ ★ — provider/workload dependent

12. Alternate features / approaches

There isn't a **direct replacement** for the KDF API because it fills a specific missing abstraction in the Java Cryptography Architecture.

However, alternatives exist depending on the requirement:

Requirement	KDF API	Alternative	When alternative is appropriate
HKDF	★ ★ ★ ★ ★	Third-party crypto provider/library	When Java version/provider doesn't provide the required KDF.
Password-based key derivation	★ ★ ★ ★	PBKDF2	When deriving a key from a password and compatibility with PBKDF2 is required.
Password hashing	★ ★ ★ ★	Argon2 / scrypt	When storing/verifying passwords.
Shared secret	★ ★ ★	KeyAgreement	When the primary

establishment			operation is Diffie-Hellman/ECDH-style key agreement.
Random symmetric key	★ ★	KeyGenerator	When you need a random key rather than a derived key.
Hashing	★	MessageDigest	When you need a digest, not key derivation.
Authentication	★ ★	Mac	When you need message authentication.
Encryption	★	Cipher	When you need encryption/decryption.
KEM shared secret	★ ★ ★ ★ ★	KEM + KDF	KEM establishes secret material; KDF converts it into protocol keys.

13. KDF vs PBKDF2 vs HKDF

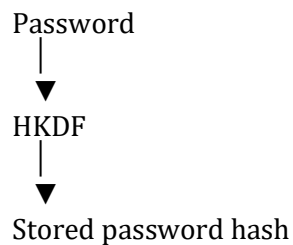
This distinction is particularly important.

Characteristic	KDF API	HKDF	PBKDF2
Category	API abstraction	Specific KDF	Password-based KDF
Primary purpose	General key derivation	Derive keys from key material	Derive keys from passwords
Designed for passwords?	Depends on algorithm	✗ No	✓ Yes
Designed for key separation?	✓	✓ Excellent	Possible, but not primary purpose
RFC	API supports standards such as HKDF	RFC 5869	RFC 8018
Typical input	Secret key material	High-	Password + salt

		entropy/appropriate key material	
Typical use	Protocols, key schedules	Session/protocol keys	Password-derived keys
Java 24 KDF API	✓	✓ Supported through API	Algorithm/provider dependent
Best use	Standardized KDF abstraction	Modern protocol key derivation	Password-based derivation

14. Important limitation: KDF is not password storage

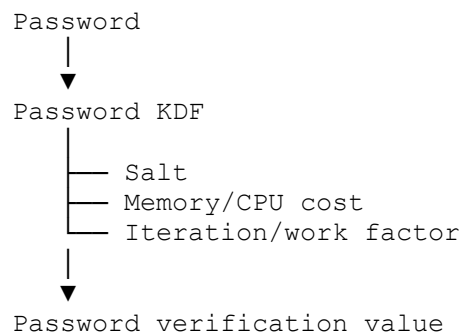
A common mistake is:



That is generally **not the right password-storage architecture**.

Password storage requires a password hashing/KDF scheme specifically designed to resist offline password guessing, with an appropriate salt and cost parameters.

For example:



So:

HKDF is excellent for key derivation from suitable secret material, but it should not automatically be treated as a password-hashing solution.

15. Java 24 Preview status

Item	Java 24 status
JEP	478
Feature	Key Derivation Function API
Status	Preview
Package	javax.crypto
Main class	KDF
HKDF support	✓
Provider SPI	KDFSpi
Preview enabled by default?	✗
Requires --enable-preview?	✓
API guaranteed unchanged?	✗
Suitable for experimentation	✓
Suitable for production where preview APIs are prohibited	✗

Oracle's Java 24 documentation explicitly identifies KDF and KDFSpi as preview APIs. ([Oracle Docs](#))

Important current-history note: the API was proposed for finalization in **Java 25 as JEP 510**, with the proposal stating that the Java 24 preview API would be finalized without change. ([OpenJDK Bug System](#))

16. Final recommendation

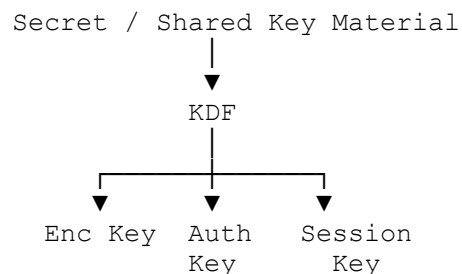
Question	Recommendation
Need to derive multiple keys from a secret?	✓ Use KDF

Implementing HKDF?	✓ Excellent use case
Building secure session protocols?	✓ Excellent use case
Implementing KEM/HPKE-style protocols?	✓ Excellent use case
Need key separation?	✓ Excellent use case
Need ordinary hashing?	✗ Use MessageDigest
Need encryption?	✗ Use Cipher
Need random keys?	✗ Use KeyGenerator
Need Diffie-Hellman/ECDH?	✗ Use KeyAgreement first
Need password storage?	✗ Use an appropriate password KDF
Need a Java 24 production API without preview dependencies?	☐ Avoid the Java 24 preview API
Developing against Java 24 and evaluating future crypto architecture?	✓ Good candidate

Bottom line

The Key Derivation Function API fills an important gap in Java's standard cryptographic APIs.

Its role is:



The biggest advantages are **standardization, key separation, provider integration, HKDF support, and cleaner cryptographic protocol implementations**. The biggest disadvantage in **Java 24 specifically** is that the API is a **preview feature**, requiring preview enablement and carrying compatibility risk. ([OpenJDK Bug System](#))

For a **new Java 24 cryptographic protocol implementation**, KDF is highly relevant; for ordinary application encryption, hashing, or password storage, use the more appropriate existing cryptographic abstraction instead.

Java 24 — Remove Windows 32-bit x86 Port

Java Version: 24

Year: 2025

Feature: Remove Windows 32-bit x86 Port

JEP: 479 — Remove Windows 32-bit x86 Port

UniqueClassName: RemoveWindows32BitX86

Important: This is not an application-level Java feature that you explicitly enable or call. It is a **platform-support removal**. Java 24 no longer provides an official Windows 32-bit (x86) JDK/JRE port. The Windows platform supported by the JDK is now **64-bit Windows on x64 (amd64)**.

1. What was removed?

Before Java 24, OpenJDK included a Windows 32-bit x86 port:

```
Windows
├── x86 / 32-bit      ← removed in Java 24
└── x64 / amd64      ← supported
```

Starting with Java 24:

```
Windows
└── x64 / amd64      ← supported
```

The change removes the Windows 32-bit x86 port from OpenJDK because 32-bit Windows has become increasingly uncommon and maintaining another architecture increases the cost and complexity of the JDK.

2. Advantages and disadvantages

Aspect	Advantages	Disadvantages
Maintenance	OpenJDK developers no longer need to maintain and test a separate	Applications still running 32-bit Windows cannot use the official Java 24 Windows port.

	Windows x86 port.	
Development resources	Engineering effort can focus on actively used 64-bit platforms.	Legacy environments may require an older JDK or migration.
Testing	Reduces the number of platform combinations that need testing.	Organizations with old 32-bit systems lose a supported deployment option.
JDK complexity	Simplifies the platform-specific implementation and build infrastructure.	Migration can be difficult for legacy hardware/software.
Modern hardware	Aligns Java with modern Windows hardware, which is predominantly 64-bit.	Old 32-bit hardware is excluded from Java 24.
Memory addressing	64-bit JVMs can address much larger memory spaces.	A 32-bit application cannot simply be upgraded to Java 24 on the same 32-bit Windows OS.
Performance potential	64-bit JVMs are better suited to modern workloads and large heaps.	64-bit applications can have larger pointer/reference overhead in some situations.
Ecosystem	Simplifies support for modern Java libraries and tooling.	Old native libraries compiled only for x86 may require migration.
Deployment	Fewer JDK binaries/platform variants need to be managed.	Deployment pipelines targeting 32-bit Windows need to be changed.
Security	Enables organizations to move away from increasingly obsolete platforms.	Legacy environments may be forced into an upgrade before they are otherwise ready.
Long-term support	Makes Java platform support more focused and sustainable.	Organizations depending on legacy Windows x86 need additional migration planning.
Containers/servers	Modern server environments are overwhelmingly 64-bit.	Very old embedded/desktop systems may be affected.

3. When to use?

The terminology is slightly different here because **you don't "use" the removal.**

You use **Java 24 on supported 64-bit Windows.**

Situation	Use Java 24?	Recommendation
Windows 11 x64	✓ Yes	Normal Java 24 deployment
Windows 10 x64	✓ Yes, subject to OS/JDK support requirements	Use 64-bit JDK
Modern Intel/AMD PC	✓ Yes	Use Windows x64 JDK
Modern enterprise desktop	✓ Yes	64-bit is the normal deployment target
Modern Windows server	✓ Yes	Use x64 JDK
Development workstation	✓ Yes	Prefer 64-bit JDK
Large Java application	✓ Yes	64-bit JVM is strongly preferable
Application requiring large heap	✓ Yes	64-bit JVM is appropriate
Windows 32-bit x86	✗ No Java 24 Windows x86 port	Upgrade OS/hardware or remain on an older supported JDK
Legacy 32-bit hardware	✗	Plan hardware/OS migration
32-bit-only native dependency	□	Obtain a 64-bit version or migrate the native integration
Legacy desktop software	□	Test compatibility before upgrading

4. When not to use Java 24

Situation	Why not use Java 24?	Alternative
32-bit Windows operating system	Java 24 does not provide the Windows x86 port.	Upgrade to 64-bit Windows or use an older supported Java release.
32-bit-only hardware	A 64-bit JVM cannot run on	Upgrade hardware.

	32-bit hardware.	
32-bit native DLL dependency	A 64-bit JVM cannot directly load a 32-bit native DLL.	Obtain/build a 64-bit DLL.
Legacy application certified only for 32-bit Java	Vendor/application compatibility may be a constraint.	Stay on the vendor-supported JDK until migration is possible.
Old Windows environment that cannot be upgraded	Java 24 is not a suitable platform choice.	Use an appropriate older supported JDK while planning migration.
Embedded/industrial legacy system	Hardware/software may depend on 32-bit Windows.	Use a supported legacy Java version or modernize the platform.

5. Real-time use cases

Real-world scenario	Before Java 24	Java 24 impact	Recommended action
Enterprise Windows desktop	Could use 32-bit or 64-bit JDK	32-bit JDK option removed	Move to 64-bit JDK
Modern Windows development machine	x86 and x64 possible	x64 is the normal choice	✔ Use Java 24 x64
Large Swing application	Could run with 32-bit Java	Requires 64-bit Java 24 on Windows	Test and migrate
Java-based accounting software	Legacy deployments may use 32-bit Windows	Java 24 x86 unavailable	Upgrade deployment platform
Manufacturing application	Some industrial PCs may still use 32-bit Windows	Java 24 cannot be deployed as Windows x86	Keep supported legacy JDK or upgrade hardware
Point-of-sale system	Older POS hardware may use 32-bit Windows	Java 24 x86 unavailable	Modernize hardware/OS
Java IDE workstation	Could theoretically use x86 JDK	Java 24 requires x64	Use 64-bit JDK

Enterprise middleware server	Usually already 64-bit	Minimal impact	Upgrade normally
Build server	Usually 64-bit	Minimal impact	Use 64-bit Java 24
Legacy native integration	May depend on x86 DLL	64-bit JVM cannot load x86 DLL	Obtain 64-bit native library
Large-memory analytics application	32-bit JVM was inherently limiting	Java 24 aligns with 64-bit architecture	✓ Strong candidate

6. Example — 32-bit Windows application

Imagine a company has:

Old PC

- ├── Windows 10 32-bit
- ├── Java 8 32-bit
- ├── Accounting Application
- └── Native DLL (32-bit)

They want:

Java 24

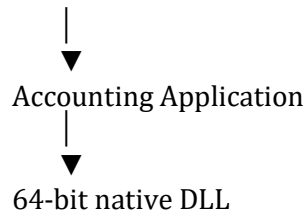
The migration cannot simply be:

Java 8 x86
↓
Java 24 x86

because the Java 24 Windows x86 port has been removed.

The migration needs to look more like:

Windows 10 32-bit
↓
▼
Upgrade hardware/OS
↓
▼
Windows 64-bit
↓
▼
Java 24 x64



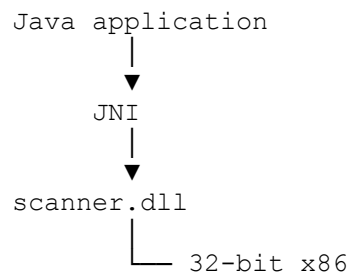
If the application doesn't use native code, migration is generally simpler.

7. Example — JNI/native library problem

Consider:

```
public class Scanner {  
    static {  
        System.loadLibrary("scanner");  
    }  
  
    public native void scan();  
}
```

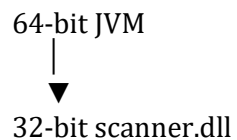
Suppose:



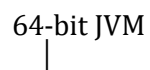
Moving to:

Java 24 x64

does **not** allow:



The native library architecture must match the JVM process architecture:



▼
64-bit scanner.dll

This becomes especially important when combined with the Java 24 **Prepare to Restrict the Use of JNI** changes you asked about previously.

8. Benefits to modern Java applications

Modern requirement	Benefit from removing x86
64-bit development	★ ★ ★ ★ ★
Large heap	★ ★ ★ ★ ★
Modern hardware	★ ★ ★ ★ ★
Simplified platform support	★ ★ ★ ★ ★
Reduced OpenJDK maintenance	★ ★ ★ ★ ★
Modern native libraries	★ ★ ★ ★
Cloud/server deployment	★ ★ ★ ★ ★
Container deployment	★ ★ ★ ★ ★
Legacy Windows compatibility	★
Old embedded hardware	★

9. 32-bit vs 64-bit Java

Characteristic	Windows x86 Java	Windows x64 Java
Address space	Limited	Very large
Large heap support	✗ Limited	✓ Excellent
Modern Windows hardware	□ Legacy	✓ Standard
Java 24 Windows port	✗ Removed	✓ Supported
Modern server	□ Poor fit	✓ Excellent

workloads		
Large enterprise applications	☐ Limited	✓ Recommended
Legacy compatibility	★ ★ ★ ★ ★	★ ★ ★
Future platform direction	☐ Declining	☐ Strong
Native x86 DLL compatibility	✓	✗
Native x64 DLL compatibility	✗	✓

10. Alternate features / alternatives

There isn't a direct **replacement feature** for "Remove Windows 32-bit x86 Port" because this JEP removes a platform port rather than adding an API.

The practical alternatives are:

Requirement	Java 24 Windows x64	Alternative	When alternative makes sense
Run Java on modern Windows	★ ★ ★ ★ ★	Java 24 x64	Recommended choice
Run Java on 32-bit Windows	✗	Older supported JDK with Windows x86	Only while legacy platform remains necessary
Keep legacy 32-bit application	✗	Continue supported legacy Java release	Until hardware/OS migration is possible
Run legacy Java application	★ ★	Upgrade application to 64-bit	Preferred long-term solution
32-bit native library	✗	64-bit version of native library	Required for Java 24 x64
Native interoperability	★ ★ ★ ★	FFM API	Consider for new native integrations

Cross-platform application	★ ★ ★ ★ ★	64-bit Java across supported OSes	Simplifies modern deployment
No native dependencies	★ ★ ★ ★ ★	Pure Java	Best for portability

11. Java 24 migration decision table

Your current environment	Java 24 migration
Windows x64 + Java 17	☐ Recommended
Windows x64 + Java 21	☐ Straightforward to evaluate
Windows x64 + Java 23	☐ Straightforward
Windows x64 + Java 8	☐ Possible, but test application compatibility
Windows x86 + Java 8	☐ Platform migration required
Windows x86 + Java 11	☐ Platform migration required
Windows x86 + Java 17	☐ Platform migration required
32-bit hardware	☐ Hardware migration required
64-bit OS + 32-bit native DLLs	☐ Native library migration required
64-bit OS + pure Java application	☐ Simplest migration case

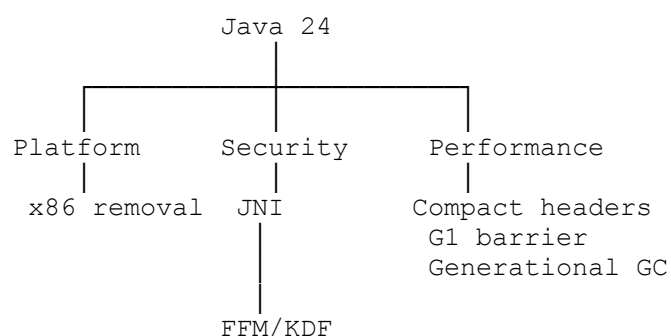
12. Relationship with other Java 24 features

This JEP fits into a broader Java modernization strategy:

Java 24 feature	Area	Relationship
Remove Windows 32-bit x86 Port	Platform	Removes obsolete 32-bit Windows support
Prepare to Restrict JNI	Security	Tightens control over native-code access

Compact Object Headers	JVM memory	Improves object memory efficiency
Generational Shenandoah	GC	Improves generational garbage collection
Late Barrier Expansion for G1	JIT/GC	Improves G1 barrier compilation
KDF API	Security	Modernizes cryptographic key derivation

The combination is significant:



13. Final recommendation

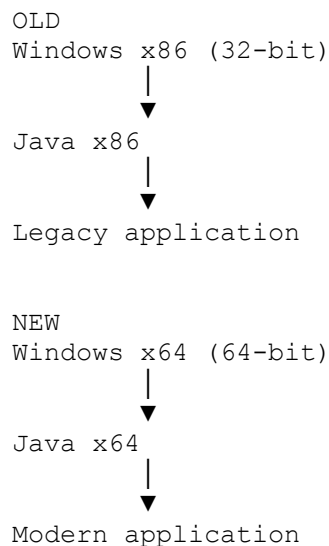
Question	Recommendation
Is this a Java API?	✗ No
Is this a JVM/platform change?	✓ Yes
Can I enable/disable it with a Java flag?	✗ No
Does Java 24 support Windows 32-bit x86?	✗ No
Does Java 24 support Windows x64?	✓ Yes
Should modern Windows deployments use x64?	✓ Yes
Do 32-bit Windows applications need migration?	✓ Yes
Do 32-bit JNI DLLs need migration?	✓ Yes
Is this important for new Java applications?	🔲 Low concern — target 64-bit

Is this important for legacy applications?	☑ Very important
Is there a direct replacement JEP?	✗ No
Best long-term alternative?	64-bit Windows + 64-bit JDK

Bottom line

JEP 479 is a cleanup and modernization change rather than a programming feature.

The fundamental transition is:



For a **new Java 24 application**, there is essentially nothing special to do: **target a 64-bit Windows environment**.

For a **legacy 32-bit Windows application**, this JEP is a migration signal: move to **64-bit Windows + 64-bit Java + 64-bit native libraries** where native integrations exist. If the old platform cannot be upgraded, Java 24 is not the appropriate Windows JDK for that environment.

Java 24 — Ahead-of-Time Class Loading & Linking

Java Version: 24

Year: 2025

Feature: Ahead-of-Time Class Loading & Linking

JEP: 483 — Ahead-of-Time Class Loading & Linking

UniqueClassName: AOTClassLoadingLinking

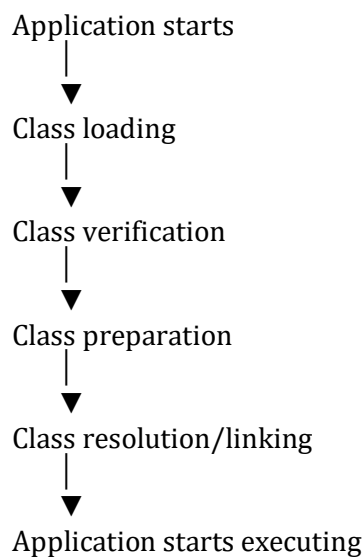
This feature introduces **AOT class loading and linking** to reduce Java application's **startup time**.

The key idea is to perform expensive class-loading and linking work **during a training/assembly phase**, save the resulting class metadata into an **AOT cache**, and reuse it when the application starts.

It is important to distinguish this from compiling the entire Java application to native code. **AOT Class Loading & Linking does not turn your Java application into a native executable**. It primarily caches class metadata and linking results.

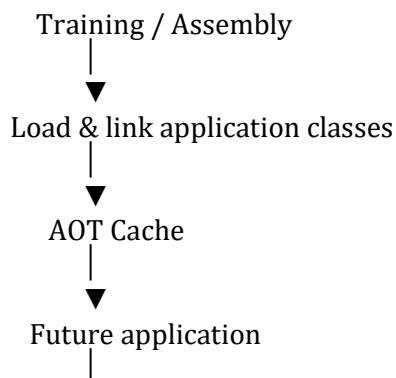
1. What problem does it solve?

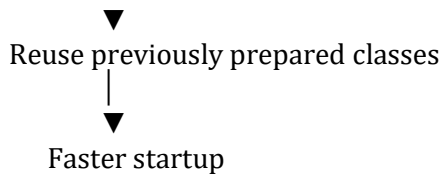
Normally, when a JVM starts an application:



For a large application, thousands of classes may need to be loaded and linked before the application can do useful work.

AOT Class Loading & Linking changes the startup process:





So the primary objective is:

Move repeatable class-loading and linking work from application startup to an earlier phase.

2. Advantages and disadvantages

Aspect	Advantages	Disadvantages
Startup time	Can substantially reduce the time required to start applications that load many classes.	Benefit depends on how much class loading/linking work can be reused.
Class loading	Previously prepared class metadata can be reused.	The cache is tied to a compatible runtime/application environment.
Class linking	Avoids repeating some expensive linking operations at every startup.	Not every startup activity can be eliminated.
Microservices	Particularly valuable for applications that start frequently.	Long-running services may gain little overall benefit after the first startup.
Serverless	Faster startup can reduce cold-start latency.	Cache creation and management introduce additional deployment complexity.
Containers	Useful when many identical containers start from the same application image.	Cache must correspond to the appropriate application/runtime configuration.
Cloud scaling	Helps instances become ready faster during scale-out.	Cache lifecycle needs to be integrated into deployment pipelines.
Developer experience	Faster repeated application startup can improve development cycles.	Training/assembly adds an additional step.

CPU usage at startup	Reduces repeated startup work.	Cache generation itself consumes CPU resources.
Memory	Can reduce some repeated startup work and potentially improve startup efficiency.	AOT cache itself consumes storage/memory resources.
Consistency	Identical application deployments can reuse the same prepared metadata.	Changes to classes, JDK, class path, or configuration can invalidate the cache.
JIT compatibility	Works alongside normal JIT compilation.	It is not a substitute for JIT compilation.
Application changes	Generally requires little/no source-code modification.	Build/deployment process may need additional AOT-cache generation.
JVM compatibility	Uses standard JVM mechanisms rather than requiring a separate native runtime.	Cache compatibility constraints must be respected.
Production deployment	Can make frequently started services more responsive.	Operational teams need to manage cache generation, storage, and invalidation.

3. AOT Class Loading & Linking vs native compilation

This distinction is extremely important.

Feature	AOT Class Loading & Linking	Native Image / native compilation
Main objective	Faster JVM startup	Produce native executable
Java bytecode retained?	✓ Yes	Usually transformed into native executable
JVM required?	✓ Yes	✗ Typically no traditional JVM at runtime

Class loading optimization	✓ Yes	Different model
Class linking optimization	✓ Yes	Mostly performed ahead of time
JIT available afterward?	✓ Yes	✗ Traditional JIT generally isn't used
Dynamic Java behavior	⚡ High	More constrained
Startup improvement	⚡ Significant potential	⚡ Very high potential
Runtime optimization	✓ VM JIT continues	Mostly ahead-of-time
Best for	JVM applications with repeated startup	Small/fast-start native deployments

Therefore:

AOT Class Loading & Linking is an optimization for JVM applications, not a replacement for the JVM.

4. When to use

Situation	Use?	Why
Microservices that start frequently	✓ Yes	Startup latency matters when instances are frequently created.
Serverless applications	✓ Strong candidate	Cold-start time directly affects request latency.
Cloud auto-scaling	✓ Strong candidate	New instances need to become ready quickly.
Large Spring applications	✓ Strong candidate	Frameworks can load thousands of classes during startup.
Large enterprise applications	✓ Consider	Startup class-loading/linking can be substantial.
Containerized applications	✓ Yes	Identical containers can reuse prepared application metadata.

Development environments	🔗 Potentially	Faster repeated startup can improve developer productivity.
CLI application launched repeatedly	🔗 Potentially	Startup cost occurs every time.
Long-running JVM	🔗 Benchmark	Startup is paid only once, so runtime performance is usually more important.
High-scale cloud service	✓ Potentially	Small startup improvements multiplied across many instances can be valuable.
Applications with huge dependency graphs	✓ Good candidate	Many classes may need loading/linking before startup completes.
Static/immutable deployment	✓ Excellent candidate	AOT cache is easier to generate and reuse consistently.

5. When not to use

Situation	Why not?	Alternative
Application runs for weeks/months	Startup cost is insignificant compared with total runtime.	Focus on runtime/JIT/GC optimization.
Tiny Java application	There may be almost no startup work worth caching.	Standard JVM startup
Application changes on every deployment/run	AOT cache reuse becomes less effective.	Normal class loading
Highly dynamic class loading	Dynamic behavior may reduce the benefit of preloading/linking.	Standard JVM class loading
Frequent dependency changes	Cache regeneration may become operationally expensive.	Normal startup
Startup isn't your bottleneck	Optimizing startup won't improve the dominant	Optimize actual bottleneck

	performance problem.	
Native executable is required	AOT class loading/linking still requires the JVM.	GraalVM Native Image / another native compilation approach
Cache cannot be reliably distributed	Deployment complexity can outweigh startup benefit.	Standard JVM startup
Very small containers	Additional cache management/storage may not justify the benefit.	Standard JVM

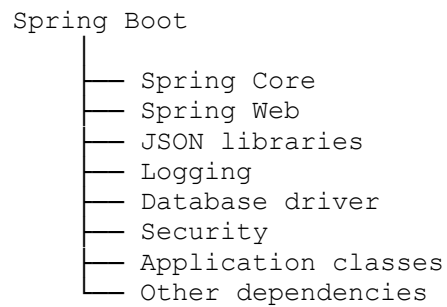
6. Real-time use cases

Real-world use case	Startup problem	How AOT Class Loading & Linking helps
Serverless REST API	New instances must start during traffic spikes.	Reuses prepared class-loading/linking information.
Spring Boot microservice	Hundreds/thousands of framework classes may be loaded.	Reduces repeated startup class-loading/linking work.
Kubernetes autoscaling	New pods must become ready quickly.	Can reduce JVM startup overhead.
E-commerce checkout service	Traffic spikes create new service instances.	Faster readiness can reduce scale-out latency.
Payment service	Startup latency can affect service availability during scaling.	Reduces repeatable JVM initialization work.
Batch workers	Workers may start and terminate frequently.	Makes repeated JVM startup cheaper.
CI/CD build workers	JVM processes can be created repeatedly.	Potentially reduces startup overhead.
Command-line tools	Application may run for only seconds.	Startup can represent a significant percentage of total execution time.
IDE plugins/tools	JVM startup happens repeatedly during tooling	Can improve startup responsiveness.

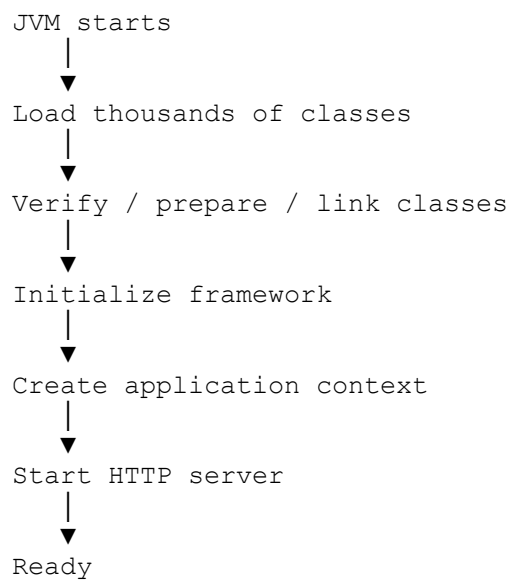
	operations.	
Cloud-native Java service	Large dependency graph + frequent scaling.	Particularly suitable for reusable AOT cache.

7. Example — Spring Boot microservice

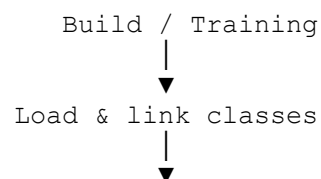
Consider a typical service:

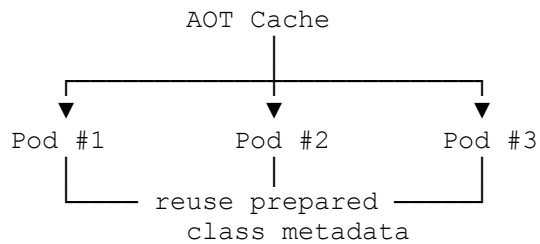


During startup:



With AOT Class Loading & Linking:





The application still executes on the JVM and still performs the work that **cannot safely or usefully be done ahead of time**.

8. Example — Kubernetes autoscaling

Imagine:

Normal traffic



3 Pods

Traffic suddenly increases:

Traffic spike



Kubernetes HPA



10 new Pods



JVM startup × 10

If each JVM spends substantial time loading/linking the same application classes:

Without AOT cache

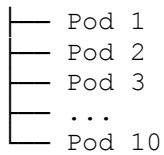
```

Pod 1 → load/link
Pod 2 → load/link
Pod 3 → load/link
...
Pod 10 → load/link
  
```

With reusable AOT class-loading/linking data:

```

AOT cache
|
  
```



The repeated class-loading/linking work can be reduced.

This is particularly attractive when **scale-out frequency is high and startup latency affects availability**.

9. Example — command-line application

Suppose:

```
java -jar report-generator.jar
```

takes:

```
Startup + class loading/linking = 800 ms
Actual report generation       = 500 ms
Total                          = 1300 ms
```

Startup is:

$$800 / 1300 \approx 61.5\%$$

So optimizing application startup can have a substantial effect on total execution time.

If AOT class-loading/linking reduces startup overhead:

```
Startup + class loading/linking = 300 ms
Actual report generation       = 500 ms
Total                          = 800 ms
```

The exact numbers are illustrative; actual improvements depend on the application.

10. What it does not optimize

Area	AOT Class Loading & Linking
Class loading/linking startup	✓ Primary target

JVM startup	✓ Potentially improves
JIT compilation	✗ Not its primary purpose
Garbage collection	✗
Database queries	✗
Network latency	✗
Algorithm complexity	✗
Application business logic	✗
Runtime CPU performance	⌘ Indirectly, but not primary
Memory leaks	✗
SQL performance	✗
Native code performance	✗

So if your application has:

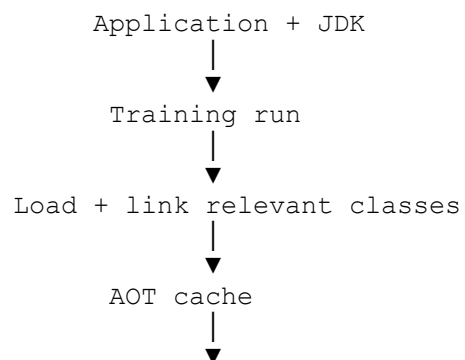
Startup = 2 seconds

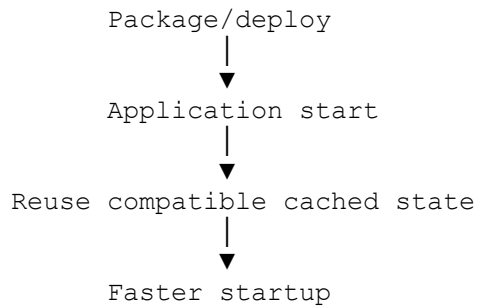
Database query = 20 seconds

AOT class-loading/linking won't solve the 20-second database problem.

11. AOT Cache lifecycle

A simplified lifecycle is:





The cache must be used with a **compatible environment**. If important application classes, dependencies, or runtime characteristics change, the cache may need to be regenerated.

12. Advantages for different applications

Application type	Expected value
Serverless	★★★★★
Frequently restarted microservice	★★★★★
Kubernetes autoscaling	★★★★★
Large Spring application	★★★★★
Cloud batch worker	★★★★
CLI tool	★★★★
Development application	★★★
Long-running application	★★
Tiny Java application	★
CPU-bound long-running application	★
Database-bound application	★
Memory-bound application	★★

13. Alternate features / approaches

There is no single direct replacement because different technologies attack different parts of Java startup.

Requirement	AOT Class Loading & Linking	Alternative	When alternative is better
Reduce JVM class-loading/linking startup	★★★★★	CDS / AppCDS	When standard class-data sharing is sufficient.
Reduce application startup	★★★★★	AOT compilation	When more aggressive startup optimization is required.
Extremely fast startup	★★★★	GraalVM Native Image	When a native executable is acceptable and dynamic JVM behavior can be constrained.
Share class metadata between JVMs	★★★★	Class Data Sharing (CDS)	Useful for multiple JVMs on the same system.
Reduce Spring startup	★★★★★	Spring AOT	When using Spring and willing to use framework-level AOT processing.
Reduce cold starts	★★★★★	CRaC	When checkpoint/restore is suitable for the deployment architecture.
Improve runtime performance	★★	JIT compilation	When startup isn't the problem and steady-state performance matters.
Reduce application startup code	★★★	Lazy initialization	When some components don't need to initialize immediately.
Reduce dependency overhead	★★★	Dependency minimization	When excessive libraries/classes are causing startup overhead.

14. AOT Class Loading & Linking vs CDS

This is one of the closest alternatives.

Characteristic	AOT Class Loading & Linking	CDS
Main objective	Faster startup through preloading/linking	Share class metadata
Class loading optimization	✓	✓
Class linking optimization	✓	Some sharing benefits
Application-specific	✓	Can be application-specific with AppCDS
Training/assembly	✓	✓ Depending on configuration
Runtime JVM	✓	✓
JIT continues afterward	✓	✓
Native executable	✗	✗
Best use	Large application startup optimization	General JVM/class metadata sharing

A useful mental model is:

CDS



Share class metadata

AOT Class Loading & Linking



Prepare more startup class state ahead of time

15. AOT Class Loading & Linking vs GraalVM Native Image

Characteristic	AOT Class Loading & Linking	GraalVM Native Image
JVM required	✓	✗
Java bytecode/JVM model	✓ Retained	Different execution model
Startup	⚡ Faster	⚡⚡ Very fast
JIT	✓ Yes	✗ Traditional JIT absent
Dynamic class loading	⚡ More flexible	⚡ Requires configuration/analysis
Reflection	⚡ JVM behavior	⚡ May require configuration depending on use
Runtime compatibility	★★★★★	★★★
Build complexity	★★★	★★
Native binary	✗	✓
Best for	Faster JVM startup	Native cloud/serverless deployment

16. AOT Class Loading & Linking vs CRaC

Another interesting alternative for startup-heavy applications is **Coordinated Restore at Checkpoint (CRaC)**.

Characteristic	AOT Class Loading & Linking	CRaC
Main objective	Optimize class loading/linking	Restore an already initialized JVM/application state
Startup improvement	★★★★★	★★★★★
JVM retained	✓	✓
Application state checkpoint	✗	✓
Class loading pre-work	✓	Indirectly

Runtime initialization pre-work	Limited	✓ Extensive
Deployment complexity	☆☆	☆☆☆
Dynamic resources	Easier	More complicated
Best use	Repeated JVM startup	Very fast restoration of complex initialized applications

17. Practical decision table

Your requirement	Recommended technology
Standard JVM + better class-loading startup	AOT Class Loading & Linking
Shared class metadata	CDS/AppCDS
Large Spring Boot startup	AOT Class Loading & Linking + framework optimizations
Extremely fast native startup	GraalVM Native Image
Restore fully initialized application	CRaC
Runtime throughput	JIT + application optimization
Reduce startup dependency cost	Remove unnecessary dependencies/lazy initialization
Serverless Java	AOT Class Loading & Linking / Native Image / CRaC, benchmarked against workload

18. Final recommendation

Question	Recommendation
Does it require application source-code changes?	Usually ✗

Does it produce a native executable?	✗
Does the JVM still run?	✓
Does JIT compilation still operate?	✓
Primary goal?	🔗 Faster startup
Does it optimize class loading?	✓
Does it optimize class linking?	✓
Best for frequently started applications?	✓
Best for serverless?	★★★★★
Best for autoscaling microservices?	★★★★★
Useful for long-running applications?	🔗 Limited
Replacement for G1/ZGC/Shenandoah?	✗
Replacement for GraalVM Native Image?	✗
Closest traditional alternative?	CDS/AppCDS
Should you benchmark it?	✓ Absolutely

Bottom line

The simplest way to remember **AOT Class Loading & Linking** is:

Traditional JVM

```

Start JVM
  ↓
Load classes
  ↓
Link classes
  ↓
Run application

```

Java 24 AOT Class Loading & Linking

```

Training / assembly
  ↓

```

```
Load + link classes
↓
Save reusable AOT cache
↓
Deploy application
↓
Start JVM
↓
Reuse prepared class state
↓
Run application faster
```

Best-fit scenario:

Large Java application + expensive startup + frequent JVM/container creation = strong candidate for AOT Class Loading & Linking.

For a long-running enterprise service that starts once and runs for weeks, the feature is much less important. For **serverless, Kubernetes autoscaling, microservices, batch workers, and frequently launched Java tools**, startup optimization can have a much larger practical impact.

Java 24 — Class-File API

Java Version: 24

Year: 2025

Feature: Class-File API

JEP: 466 — Class-File API (Second Preview)

UniqueClassName: ClassFileAPI

Important: In Java 24, the Class-File API is a **second-preview API**. It provides a standard API for **reading, writing, and transforming Java .class files**. It is intended to eventually replace lower-level third-party bytecode manipulation libraries for many use cases.

The API is in the `java.lang.classfile` package.

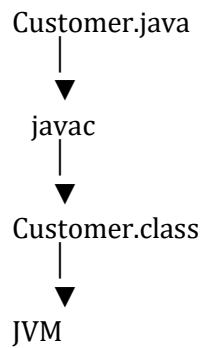
1. What is the Class-File API?

A Java source file:

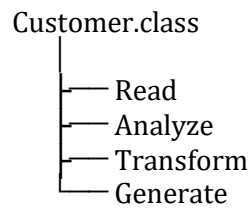
```
class Customer {
    void save() {
        System.out.println("Saving");
    }
}
```

}

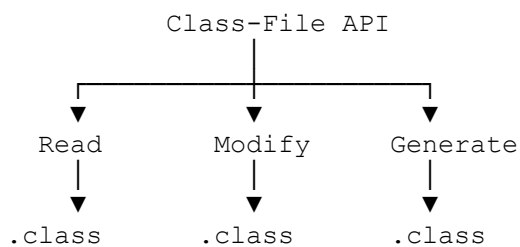
is compiled into:



The Class-File API works directly with the .class representation:



Conceptually:



It provides a **structured, modern Java API for class-file manipulation**, while taking care of many low-level class-file details.

2. Advantages and disadvantages

Aspect	Advantages	Disadvantages
Standard API	Comes from the JDK rather than requiring a third-party bytecode library.	Java 24 version is still a preview API.
Bytecode manipulation	Provides APIs to inspect and transform class files.	Bytecode manipulation remains technically complex.

Class generation	Can generate class files programmatically.	Developers need knowledge of JVM class-file concepts.
Class transformation	Useful for modifying classes during build or runtime instrumentation.	Incorrect transformations can produce invalid classes or runtime failures.
JVM evolution	Designed with future class-file evolution in mind.	Preview API details can change between Java releases.
Type safety	Higher-level abstractions are safer than manually constructing raw class-file bytes.	It cannot completely prevent logical bytecode-generation mistakes.
Performance	Designed as a modern JDK API with efficient class-file processing.	Performance should still be benchmarked for large-scale instrumentation.
Dependencies	Can eliminate dependencies on libraries such as ASM for certain use cases.	Existing projects may already be deeply integrated with mature bytecode libraries.
Tooling	Excellent foundation for bytecode analysis and transformation tools.	Building sophisticated tooling requires significant JVM knowledge.
Frameworks	Can support framework-level instrumentation and code generation.	Framework migration from existing bytecode libraries can be substantial.
Build-time processing	Useful for modifying/generated classes during builds.	Adds complexity to build pipelines.
Runtime agents	Can be used in instrumentation scenarios.	Runtime transformation must be carefully tested because it affects executing application code.
Maintenance	Standard JDK API can reduce long-term dependency management.	Preview APIs require migration when the API evolves.
Class-file versions	Designed to work with modern JVM class-file structures.	Supporting many historical class-file versions can still be challenging.

Learning curve	More structured than manipulating raw class-file bytes.	Still significantly harder than ordinary Java programming.
-----------------------	---	--

3. Major API components

The central package is:

`java.lang.classfile`

Some important types include:

API	Purpose
ClassFile	Main entry point for parsing and generating class files
ClassModel	Represents a parsed class file
ClassTransform	Defines transformations applied to classes
MethodModel	Represents a method in a class
MethodTransform	Transforms method information/code
FieldModel	Represents a field
Attribute	Represents class-file attributes
CodeModel	Represents method bytecode
CodeTransform	Transforms bytecode instructions
Label	Represents bytecode branch targets
Opcodes	Represents JVM bytecode operations

The exact API surface is preview functionality in Java 24 and therefore should be checked against the specific Java 24 documentation when implementing production tooling.

4. Simple example — reading a .class file

Conceptually:

```

import java.lang.classfile.ClassFile;

import java.nio.file.Files;
import java.nio.file.Path;

public class ClassFileReader {

    public static void main(String[] args) throws Exception {

        byte[] bytes =
            Files.readAllBytes(Path.of("Customer.class"));

        ClassFile.of()
            .parse(bytes);
    }
}

```

The important difference is that you aren't manually interpreting:

- magic number
- constant pool
- access flags
- interfaces
- fields
- methods
- attributes
- bytecode

The Class-File API provides structured representations of these components.

5. Example — inspect methods

Conceptually:

```

var classFile = ClassFile.of();

var model = classFile.parse(bytes);

model.methods().forEach(method ->
    System.out.println(method.methodName()));

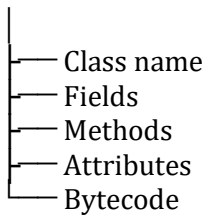
```

You can build tooling such as:

```

Customer.class
|
▼
ClassFile API

```



This is useful for **static analysis, bytecode inspection, migration tools, and framework tooling**.

6. When to use Class-File API

Situation	Use?	Why
Bytecode analysis tool	✓ Yes	Directly inspect class-file structures.
Class-file transformation	✓ Yes	Designed specifically for this purpose.
Build-time bytecode modification	✓ Yes	Can transform generated .class files.
Java agent/instrumentation	✓ Consider	Useful for modifying loaded classes.
Framework code generation	✓ Yes	Can generate JVM class files programmatically.
AOP implementation	✓ Yes	Can insert or transform bytecode.
Profiling/instrumentation	✓ Yes	Can inject monitoring instructions.
Security scanning	✓ Yes	Inspect classes without executing application code.
Class-file migration tools	✓ Yes	Analyze or rewrite classes for newer JVM versions.
Obfuscation tooling	⚠ Potentially	Class transformations can support such tooling.
Custom compiler/toolchain	✓ Yes	Useful for producing JVM class files.
Java application business logic	✗ No	Ordinary applications don't need class-file manipulation.
Simple reflection	✗ No	Use Java Reflection instead.

Normal object creation	✗ No	Use ordinary Java classes.
-------------------------------	-------------	----------------------------

7. When not to use it

Situation	Why not?	Better alternative
Normal application development	Bytecode manipulation is unnecessary.	Ordinary Java language features
Need to inspect object fields at runtime	Class-file API is overkill.	Reflection
Need to invoke methods dynamically	No need to manipulate class files.	Reflection / MethodHandles
Need dependency injection	Don't build your own bytecode engine unnecessarily.	Existing DI framework
Need simple logging	Bytecode transformation is excessive.	Logging framework
Need JSON serialization	No need for class-file manipulation.	Jackson / JSON-B / Gson, etc.
Need runtime proxies	Often easier with Proxy or MethodHandles.	Java dynamic proxy / MethodHandles
Small application	Complexity outweighs benefits.	Standard Java APIs
You need a stable Java 24 API	Class-File API is preview in Java 24.	Stable third-party library or wait for final API
Existing ASM-based system is mature and stable	Migration may provide little immediate benefit.	Continue using ASM until migration is justified

8. Real-time use cases

Real-world use case	What the Class-File API does	Example
----------------------------	-------------------------------------	----------------

APM agent	Adds timing/metrics instructions to methods	Measure execution time of service methods
Java profiler	Instruments selected methods	Track CPU usage
Code coverage	Inserts counters into bytecode	Record which branches/methods execute
AOP framework	Adds cross-cutting behavior	Inject transaction/security logic
Security scanner	Reads class structure	Detect suspicious API usage
Build optimizer	Modifies classes during build	Remove/replace generated code
Framework code generation	Generates .class files	Generate proxy/adaptor implementations
Mocking framework	Transforms classes	Replace method behavior during tests
Testing tools	Instruments classes	Add test probes
Java agent	Modifies loaded classes	Add runtime observability
Bytecode optimizer	Rewrites methods	Optimize generated code
Compatibility tool	Analyzes class-file versions	Identify classes incompatible with a target JDK
Serialization framework	Generates optimized accessors	Avoid repeated reflection overhead
ORM framework	Generates entity-related classes	Create runtime proxies
Application monitoring	Injects telemetry	Trace HTTP/database method execution

9. Example — application monitoring

Suppose you have:

```
class PaymentService {
```

```

    public void processPayment() {
        // payment logic
    }
}

```

A monitoring agent could transform it conceptually into:

```

class PaymentService {

    public void processPayment() {

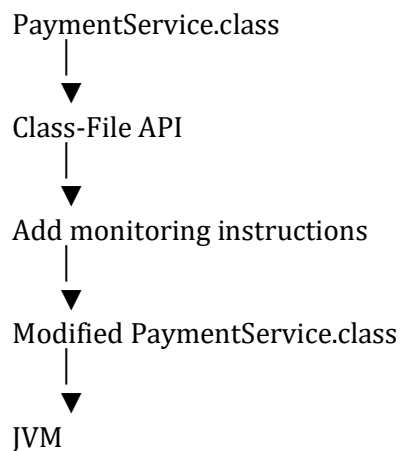
        Timer.start();

        try {
            // original payment logic
        } finally {
            Timer.stop();
        }
    }
}

```

The developer didn't have to modify the source.

The architecture becomes:



This is a classic bytecode instrumentation use case.

10. Example — code coverage

Original:

```

if (amount > 1000) {

```

```
    approve();
} else {
    reject();
}
```

Instrumentation could conceptually become:

```
Coverage.hit(1);

if (amount > 1000) {

    Coverage.hit(2);
    approve();

} else {

    Coverage.hit(3);
    reject();
}
```

At the end of the test:

```
Branch 1 → executed
Branch 2 → executed
Branch 3 → NOT executed
```

This enables coverage tools to determine which code paths were executed.

11. Example — class generation

Imagine a framework needs to generate:

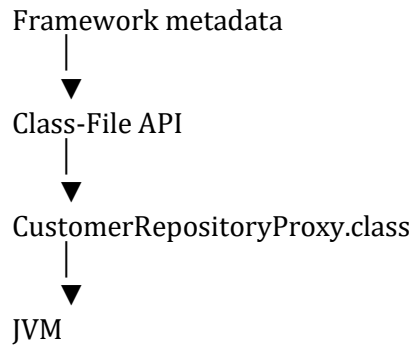
```
class CustomerRepositoryProxy
    extends CustomerRepository {

    // generated behavior
}
```

Instead of generating Java source:

```
Java source
↓
javac
↓
.class
```

a framework can generate class-file structures directly:



This can be useful for **frameworks, proxies, serializers, ORM implementations, and dependency-injection infrastructure.**

12. Class-File API vs Reflection

These are frequently confused.

Requirement	Class-File API	Reflection
Inspect .class file without loading it	★★★★★	✗
Inspect loaded class	★★★	★★★★★
Modify bytecode	★★★★★	✗
Generate .class files	★★★★★	✗
Invoke methods	★★	★★★★★
Read runtime annotations	★★★	★★★★★
Analyze bytecode	★★★★★	★
Build instrumentation tools	★★★★★	★★
Ordinary application development	★	★★★★★
Runtime object manipulation	★	★★★★★

Think:

Reflection



"What does this loaded Java object/class look like?"

Class-File API



"What is inside this .class file, and how can I transform it?"

13. Class-File API vs ASM

ASM has historically been one of the major choices for Java bytecode manipulation.

Characteristic	Class-File API	ASM
Provider	JDK	Third-party library
Standard Java API	✓	✗
Bytecode manipulation	✓	✓
Mature ecosystem	🕒 Newer	★★★★★
Java-version evolution	Designed into JDK	Library updates required
Dependency required	✗	✓
Existing tooling	Growing	Huge
Learning curve	Moderate/high	High
Framework adoption	Emerging	Extensive
Java 24 status	Preview	Stable external library
Best for new JDK-integrated tooling	★★★★★	★★★★
Best for mature existing systems	★★★	★★★★★

For a new project targeting modern Java, the Class-File API is highly interesting. For an established production system already using ASM extensively, there may be **no immediate reason to migrate everything**.

14. Class-File API vs Byte Buddy

Requirement	Class-File API	Byte Buddy
Low-level class-file manipulation	★★★★★	★★★★
High-level runtime code generation	★★★	★★★★★
Runtime instrumentation	★★★★	★★★★★
Easy proxy generation	★★★	★★★★★
Direct JDK API	✓	✗
Third-party dependency	✗	✓
Fine-grained JVM bytecode control	★★★★★	★★★★
Ease of use	★★★	★★★★★
Framework-oriented development	★★★	★★★★★
Existing ecosystem	?	★★★★★

So:

Class-File API is a class-file toolkit; Byte Buddy is a higher-level code-generation/instrumentation framework.

15. Alternate features / approaches

Requirement	Class-File API	Alternative	When alternative is better
Read class files	★★★★★	ASM	Mature ecosystem or existing infrastructure
Transform bytecode	★★★★★	ASM	Existing complex transformation pipeline

Generate classes	★★★★★	Byte Buddy	Higher-level runtime code generation
Runtime instrumentation	★★★★	Byte Buddy	Easier agent/instrumentation development
Runtime proxies	★★★	java.lang.reflect.Proxy	Simple interface-based proxy
Dynamic method invocation	★★	MethodHandles	No bytecode transformation required
Inspect loaded classes	★★★	Reflection	You only need runtime metadata
Generate source code	★★★	JavaPoet	You want Java source rather than bytecode
Compile generated source	★★	Java Compiler API	Source-level generation is more appropriate
Static bytecode analysis	★★★★★	ASM / analysis frameworks	Mature analysis infrastructure
Build-time instrumentation	★★★★★	ASM	Existing build tooling already uses ASM

16. Which alternative should you choose?

Your requirement	Recommended choice
Need standard JDK class-file manipulation	Class-File API
Building new low-level bytecode tooling	Class-File API
Existing mature ASM infrastructure	ASM
High-level runtime instrumentation	Byte Buddy
Simple interface proxy	java.lang.reflect.Proxy

Dynamic invocation	MethodHandles
Runtime metadata inspection	Reflection
Generate Java source	JavaPoet / source generator
Compile generated Java source	Java Compiler API

17. When Class-File API is a particularly good choice

Project	Suitability
Bytecode analyzer	★★★★★
Java agent	★★★★★
APM tooling	★★★★★
Code coverage tool	★★★★★
Security scanner	★★★★★
Build-time class transformer	★★★★★
Framework internals	★★★★
Proxy generator	★★★★
ORM infrastructure	★★★★
Serialization framework	★★★★
Ordinary Spring Boot business application	★
REST API implementation	★
CLI application	★
Simple desktop application	★

18. Important limitation — preview status

For Java 24:

Property	Status
JEP	466
Feature	Class-File API
Release	Java 24
Status	Second Preview
Package	java.lang.classfile
Source-code API	✓
Read class files	✓
Generate class files	✓
Transform class files	✓
Stable/final in Java 24	✗
Requires preview compilation	✓
Requires preview runtime	✓

Therefore, Java 24 usage requires preview support, for example:

```
javac --release 24 --enable-preview MyTool.java
java --enable-preview MyTool
```

For an organization that prohibits preview APIs in production, a stable third-party library such as ASM or Byte Buddy may be preferable until the API is finalized.

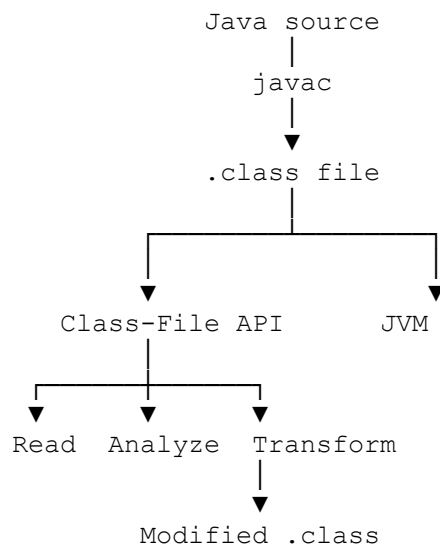
19. Final recommendation

Question	Recommendation
Need to manipulate .class files?	✓ Excellent candidate
Need to analyze bytecode?	✓ Excellent candidate

Need to generate classes?	✓ Excellent candidate
Building Java agents?	✓ Strong candidate
Building APM/profiling tools?	✓ Strong candidate
Building framework infrastructure?	✓ Strong candidate
Normal Java business application?	✗ Don't use it
Need runtime reflection?	✗ Use Reflection
Need simple dynamic proxy?	✗ Consider Proxy
Existing ASM infrastructure?	🔗 Migration isn't automatically necessary
Need high-level code generation?	🔗 Consider Byte Buddy
Need stable API specifically in Java 24?	🔗 Use caution — preview
Need future JDK-integrated bytecode tooling?	★★★★★

Bottom line

The easiest way to remember the **Class-File API** is:



Use it when you are building tools/frameworks that need to understand, generate, or transform Java bytecode. Do not use it as part of ordinary business application code.

For new low-level bytecode tooling, **Class-File API is the strategic JDK-native option**. For mature existing systems, **ASM** remains a strong alternative, while **Byte Buddy** is often better when you want higher-level runtime instrumentation and code generation rather than directly working with class-file structures.

Java 24 — Stream Gatherers

Property	Details
Java Version	24
Year	2025
Feature	Stream Gatherers
JEP	JEP 485
UniqueClassName	StreamGatherers
Status in Java 24	Final
Main API	java.util.stream.Gatherer, java.util.stream.Gatherers
Purpose	Allows developers to create custom intermediate Stream operations that can maintain state, buffer elements, emit multiple results, or perform custom transformations.

Stream Gatherers solve a limitation of the traditional Stream API: operations such as `map()`, `filter()`, and `flatMap()` are powerful, but some stateful intermediate transformations—such as sliding windows, prefix scans, or controlled concurrent mapping—are awkward to express. JEP 485 adds `Stream.gather(...)` for these cases. ([OpenJDK Mail](#))

Important

Unlike a Collector, a **Gatherer is an intermediate Stream operation**:

```
Stream
↓
map()
↓
filter()
↓
gather() ← Stream Gatherer
↓
```

```
map()
↓
toList()
```

A Gatherer can transform **one input element into zero, one, or many output elements**, and can maintain state between elements. ([OpenJDK CVM](#))

1. Advantages and Disadvantages

Advantages	Disadvantages
Allows creation of custom intermediate Stream operations	More complex than ordinary map(), filter(), etc.
Supports stateful stream processing	Stateful operations can be harder to reason about
Supports windowing operations	Large windows can consume significant memory
Supports sliding windows	Sliding windows can create many overlapping objects
Supports incremental/prefix calculations	Not every problem benefits from a Gatherer
Supports order-dependent transformations	Some gatherers cannot be efficiently parallelized
mapConcurrent() provides controlled concurrency	Concurrent processing adds complexity and resource consumption
Can integrate naturally into existing Stream pipelines	Debugging complex gatherers can be harder
Can emit results incrementally	Poorly designed gatherers can create excessive allocations
Can support custom domain-specific Stream operations	Custom Gatherers increase maintenance responsibility
Can terminate processing early	Developers must understand downstream cancellation behavior
Can potentially be parallelized when a suitable combiner is supplied	Stateful parallel Gatherers require careful design

The API specifically supports state creation, integration, optional state combination, and finalization. A Gatherer can therefore be considerably more expressive than a normal stateless intermediate operation. ([OpenJDK CVM](#))

2. What Problem Does Stream Gatherers Solve?

Before Gatherers, suppose you wanted **sliding windows**:

```
Input:
1 2 3 4 5

Window size = 3

Output:
[1, 2, 3]
[2, 3, 4]
[3, 4, 5]
```

There was no standard intermediate Stream operation specifically designed for this.

You could write custom loops or complicated Collectors, but neither is as natural as:

```
List<List<Integer>> result =
    Stream.of(1, 2, 3, 4, 5)
        .gather(Gatherers.windowSliding(3))
        .toList();
```

Result:

```
[[1, 2, 3], [2, 3, 4], [3, 4, 5]]
```

Java 24 provides standard Gatherers for common cases such as fixed windows, sliding windows, folding, scanning, and concurrent mapping. ([OpenJDK CVM](#))

3. Important Built-in Gatherers

Gatherer	Purpose	Example
windowFixed(n)	Groups elements into fixed-size batches	[1,2,3], [4,5,6]
windowSliding(n)	Creates overlapping windows	[1,2,3], [2,3,4]

fold()	Performs ordered reduction-like processing	Build a value incrementally
scan()	Produces every intermediate accumulated value	1, 3, 6, 10...
mapConcurrent()	Applies mapping concurrently with bounded concurrency	Parallel API calls

These are provided by `java.util.stream.Gatherers`. ([OpenJDK CVM](#))

4. `windowFixed()` Example

Requirement

Process records in batches of 3.

```
List<List<Integer>> batches =
    Stream.of(1, 2, 3, 4, 5, 6, 7, 8)
        .gather(Gatherers.windowFixed(3))
        .toList();
```

Result:

```
[
  [1, 2, 3],
  [4, 5, 6],
  [7, 8]
]
```

Notice that the last window can contain fewer elements than the requested window size. ([OpenJDK CVM](#))

Real-world example

```
List<List<Order>> batches =
    orders.stream()
        .gather(Gatherers.windowFixed(100))
        .toList();
```

Useful for:

- Bulk database inserts

- Batch REST requests
 - Bulk message publishing
 - Batch file processing
-

5. `windowSliding()` Example

Suppose we receive stock prices:

100, 105, 110, 108, 115

A 3-value sliding window gives:

[100,105,110]
[105,110,108]
[110,108,115]

Java:

```
List<List<Integer>> windows =  
    Stream.of(100, 105, 110, 108, 115)  
        .gather(Gatherers.windowSliding(3))  
        .toList();
```

This is particularly useful for:

- Moving averages
- Time-series analysis
- Anomaly detection
- Sensor data
- Financial analysis
- Monitoring metrics

The official API notes that large window sizes can increase temporary memory usage. ([OpenJDK CVM](#))

6. `scan()` Example

`scan()` produces the intermediate accumulated values.

```
List<Integer> result =  
    Stream.of(1, 2, 3, 4, 5)  
        .gather(  
            Gatherers.scan(  
                () -> 0,
```

```

        Integer::sum
    )
)
.toList();

```

Conceptually:

Input:
1 2 3 4 5

Output:
1 3 6 10 15

This is useful when you need **every intermediate state**, rather than only the final result.

Real-time example

Account balance changes:

```

+100
-20
+50
-10

```

Running balance:

```

100
80
130
120

```

This is a natural use case for `scan()`.

7. fold() Example

`fold()` is useful for an ordered reduction where the operation is inherently sequential.

```

String result =
    Stream.of("A", "B", "C", "D")
        .gather(
            Gatherers.fold(
                () -> "",
                (current, value) -> current + value
            )
        )
        .findFirst()
        .orElse("");

```

Result:

ABCD

`fold()` differs from ordinary `reduce()` in that `Gatherers` are intended for more general intermediate stream transformations, especially those that need custom stateful behavior. The standard `fold()` is specifically described as useful when no combiner can be implemented or the reduction is intrinsically order-dependent. ([OpenJDK CVM](#))

8. `mapConcurrent()` Example

This is one of the more interesting built-in `Gatherers`.

Suppose you need to call an external service for many customers:

```
List<CustomerDetails> result =
    customerIds.stream()
        .gather(
            Gatherers.mapConcurrent(
                10,
                customerService::loadCustomer
            )
        )
        .toList();
```

Here:

Maximum concurrency = 10

The operation executes the mapping function concurrently using virtual threads while preserving stream ordering. ([OpenJDK CVM](#))

Real-world scenario

```
1000 customer IDs
  ↓
mapConcurrent(20, API call)
  ↓
20 concurrent operations
  ↓
Customer details
```

This can be useful for I/O-bound workloads such as:

- REST API calls
- Database lookups
- Remote service calls

- File/network operations

However, concurrency should be limited according to the capacity of the downstream system.

9. When to Use Stream Gatherers

Situation	Use Gatherer?	Reason
Fixed-size batching	✓ Yes	windowFixed() is directly suited
Sliding windows	✓ Yes	windowSliding() directly models the problem
Running/partial calculations	✓ Yes	scan() is designed for incremental accumulation
Order-dependent reduction	✓ Yes	fold() can express it naturally
Controlled concurrent mapping	✓ Yes	mapConcurrent() provides bounded concurrency
Stateful intermediate processing	✓ Yes	Gatherers are specifically designed for this
Custom intermediate operation	✓ Yes	Main purpose of the API
Complex event-stream transformation	✓ Often	Can maintain state across elements
Simple element transformation	✗ Usually no	Use map()
Simple filtering	✗ No	Use filter()
Simple one-to-many transformation	✗ Usually no	Use flatMap()/mapMulti()
Simple final aggregation	✗ Usually no	Use collect()/reduce()

10. When NOT to Use Stream Gatherers

Situation	Why Avoid Gatherers?	Better Choice
Simple transformation	Gatherer is unnecessarily complex	map()
Simple filtering	No state is required	filter()
Simple aggregation	Standard Stream API is clearer	reduce()
Building a List/Map/Set	Gatherer is not intended for final collection	collect()
Very complicated business workflow	Stream pipeline may become difficult to maintain	Ordinary classes/loops
Huge sliding windows	Potential memory overhead	Specialized streaming/windowing system
Complex asynchronous workflow	Gatherer may not provide sufficient workflow semantics	CompletableFuture, structured concurrency, reactive framework
CPU-intensive parallel processing	mapConcurrent() is primarily useful for concurrent mapping; benchmark alternatives	Parallel streams/executors
Operation doesn't need Stream semantics	Gatherer adds unnecessary abstraction	Normal loop
Team unfamiliar with advanced Streams	Maintainability may suffer	Conventional Java

11. Real-Time Use Cases

Real-Time Use Case	Problem	Gatherer	Example
Bulk database insertion	Insert records in manageable batches	windowFixed()	500 records per batch
REST API batching	API accepts limited batch size	windowFixed()	100 requests/batch

Moving average	Need recent N values	windowSliding()	Last 10 stock prices
Fraud detection	Analyze recent transactions	windowSliding()	Last 5 transactions
Sensor monitoring	Analyze recent sensor readings	windowSliding()	Last 60 readings
Running account balance	Need balance after every transaction	scan()	Debit/credit stream
Cumulative metrics	Need running totals	scan()	Orders processed
Ordered aggregation	Sequential transformation required	fold()	Build ordered result
External API enrichment	Enrich thousands of records	mapConcurrent()	Customer → profile API
Microservice fan-out	Call several services concurrently	mapConcurrent()	Product → inventory service
Log processing	Group logs into batches	windowFixed()	1,000 logs/batch
Time-series processing	Analyze overlapping observations	windowSliding()	CPU metrics
ETL pipelines	Stateful transformation	Custom Gatherer	Normalize event sequence

12. Example — Batch Processing

Without Gatherer:

```
for (int i = 0; i < orders.size(); i += 100) {
    List<Order> batch =
        orders.subList(i, Math.min(i + 100, orders.size()));

    processBatch(batch);
}
```

With Gatherer:

```
orders.stream()
    .gather(Gatherers.windowFixed(100))
    .forEach(this::processBatch);
```

Benefit

The second version fits naturally into a Stream pipeline:

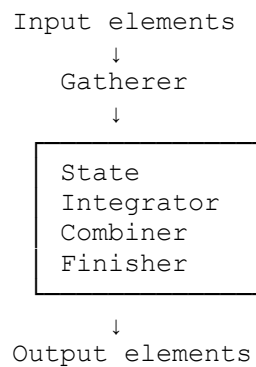
```
orders
  ↓
filter()
  ↓
map()
  ↓
gather(windowFixed(100))
  ↓
processBatch()
```

13. Custom Gatherer

The real power of JEP 485 is not limited to the predefined Gatherers.

A custom Gatherer can maintain its own state and decide when/how elements are emitted.

Conceptually:



A Gatherer can define:

Component	Responsibility
initializer()	Creates processing state
integrator()	Processes each input element

combiner()	Combines states when parallel processing is supported
finisher()	Performs final processing

The Gatherer API explicitly models these four pieces. ([OpenJDK CVM](#))

14. Gatherer vs Collector

This distinction is **very important**.

Feature	Gatherer	Collector
Position in pipeline	Intermediate	Terminal
API	gather()	collect()
Input	Stream elements	Stream elements
Output	Another Stream	Final result
Can produce intermediate results?	✓ Yes	Primarily final result
Stateful processing	✓ Yes	✓ Yes
Windowing	✓ Natural	Awkward
Prefix scan	✓ Natural	Not the primary purpose
Final aggregation	Possible	✓ Primary purpose
Example	gather(Gatherers.windowFixed(100))	collect(Collectors.toList())

Think of it as:

Collector:
Stream $\xrightarrow{\text{collect()}}$ Final Result

Gatherer:
Stream $\rightarrow$ Gatherer $\rightarrow$ Stream $\rightarrow \dots \rightarrow$ Final Result
gather()

15. Alternate Features

Requirement	Stream Gatherers	Best Alternative	When Alternative Is Better
Simple transformation	gather()	map()	Almost always use map()
Filter elements	gather()	filter()	When only filtering is required
One-to-many transformation	gather()	flatMap() / mapMulti()	When state isn't required
Final aggregation	gather()	reduce()	When only one final result is needed
Collect into collection	gather()	collect()	Standard terminal aggregation
Fixed batching	windowFixed()	Manual loop	Legacy/simple imperative code
Sliding window	windowSliding()	Custom loop/deque	When maximum control/performance is required
Running total	scan()	reduce()/manual loop	reduce() when only final total is needed
Concurrent mapping	mapConcurrent()	Virtual threads + executor	When you need custom concurrency control
Async workflows	mapConcurrent()	CompletableFuture	When asynchronous composition is required
Complex event processing	Custom Gatherer	Reactive/event-stream framework	When processing is continuous and distributed
Complex business	Custom Gatherer	Ordinary loop/classes	When readability is

logic			more important than Stream composition
-------	--	--	--

16. Stream Gatherers vs Common Alternatives

Requirement	Recommended
element → element	map()
element → 0..N elements	flatMap() / mapMulti()
element → true/false	filter()
stream → one result	reduce()
stream → collection	collect()
stream → batches	Gatherer
stream → sliding windows	Gatherer
stream → incremental results	Gatherer
stream → custom stateful transformation	Gatherer
stream → controlled concurrent mapping	Gatherer / mapConcurrent()
Complex asynchronous workflow	CompletableFuture / structured concurrency
Continuous distributed event processing	Kafka Streams / Apache Flink / similar systems

17. Performance Considerations

Consideration	Impact
Stateful processing	May require additional memory
Large fixed windows	Can increase temporary memory usage
Large sliding windows	Can produce many overlapping windows
Custom objects per window	Can increase GC pressure

mapConcurrent()	Can improve I/O throughput but consumes resources
Excessive concurrency	Can overload databases/services
Parallel Gatherer	Requires a correct/efficient combiner
Sequential stateful processing	May limit parallel scalability
Custom Gatherer	Performance depends heavily on implementation

The JDK documentation specifically warns that large window sizes may use excessive memory during evaluation. ([OpenJDK CVM](#))

18. Best-Practice Decision Table

Question	Recommendation
Can map() solve it?	Use map()
Can filter() solve it?	Use filter()
Can flatMap()/mapMulti() solve it?	Prefer them
Is the operation a final aggregation?	Use collect()/reduce()
Do you need state between stream elements?	Consider Gatherer
Do you need batches/windows?	Strong candidate for Gatherer
Do you need running intermediate results?	Use scan()
Do you need ordered folding?	Consider fold()
Do you need bounded concurrent mapping?	Consider mapConcurrent()
Do you need a custom intermediate operation?	Use custom Gatherer
Is the logic extremely complex?	Prefer conventional Java

19. Overall Advantages vs Alternatives

Aspect	Stream Gatherers	Traditional Approach
Readability for windowing	★★★★★	★★
Custom stream transformations	★★★★★	★★
Simple transformations	★★★	★★★★★
Final aggregation	★★★	★★★★★
Stateful intermediate operations	★★★★★	★★
Batch processing	★★★★★	★★★
Sliding-window processing	★★★★★	★★
Concurrent mapping	★★★★	★★★
Learning curve	★★★	★★★★★
Flexibility	★★★★★	★★★
Risk of overengineering	Medium	Low

Final Recommendation

Scenario	Recommendation
Normal map/filter/reduce pipeline	✗ Don't introduce Gatherers
Batch processing	✓ Use <code>Gatherers.windowFixed()</code>
Sliding-window calculations	✓ Use <code>Gatherers.windowSliding()</code>
Running/cumulative results	✓ Use <code>Gatherers.scan()</code>
Ordered stateful reduction	✓ Use <code>Gatherers.fold()</code>

Controlled concurrent I/O	✓ Consider <code>Gatherers.mapConcurrent()</code>
Custom stateful intermediate operation	✓ Use a custom <code>Gatherer</code>
Simple business logic	✗ Prefer ordinary Stream operations
Very complex workflow	✗ Prefer conventional classes/loops or a dedicated framework

In short: Stream `Gatherers` are best viewed as the missing “**custom intermediate operation**” capability of the Java Stream API. They are particularly valuable for **windowing, stateful transformations, incremental calculations, and controlled concurrent mapping**. For ordinary `map()`, `filter()`, `flatMap()`, `reduce()`, and `collect()` scenarios, the existing Stream API remains simpler and preferable. ([OpenJDK Mail](#))

Java 24 — Permanently Disable the Security Manager

Property	Details
Java Version	24
Year	2025
Feature	Permanently Disable the Security Manager
JEP	JEP 486
UniqueClassName	<code>DisableSecurityManager</code>
Status	Final
Nature	Platform/security change
Main objective	Permanently prevent the Java Security Manager from being enabled
Replacement	No single direct replacement

Important: This is not a new security feature that you enable. Java 24 **removes the ability to enable the Security Manager**. The Security Manager API itself is planned for removal in a future

JDK release. ([Oracle Docs](#))

1. What Is the Security Manager?

Historically, Java's Security Manager allowed an application to enforce permissions such as:

```
Can this code:
├─ read a file?
├─ write a file?
├─ open a network connection?
├─ access a system property?
└─ perform another restricted operation?
```

For example, an application could install a policy that restricted file access:

```
System.setSecurityManager(new SecurityManager());
```

and use a policy file such as:

```
grant {
    permission java.io.FilePermission
        "/tmp/app.log", "read";
};
```

However, the Security Manager was deprecated for removal in Java 17 and is **permanently disabled starting with Java 24**. ([Oracle Docs](#))

2. What Changes in Java 24?

Area	Before Java 24	Java 24
Enable Security Manager at startup	Possible in older releases	✗ Not possible
-Djava.security.manager	Could enable it	✗ JVM startup error
Custom SecurityManager	Could install one	✗ UnsupportedOperationException
System.getSecurityManager()	Could return manager	Returns null
System.setSecurityManager()	Could install manager	Throws UnsupportedOperationException
AccessController.doPrivileged()	Could change security context	Runs as if no Security Manager Exists

SecurityManager.check*()	Could perform permission checks	Throws security-related exceptions
Java security policy file	Used for Security Manager	Removed
Security Manager API	Functional	Degraded; planned for future removal
RMI remote code downloading	Supported with Security Manager	Removed
OS/container security	External to Security Manager	Recommended

Oracle explicitly documents that enabling the Security Manager is now an error and that the API will be removed in a future release. ([Oracle Docs](#))

3. Advantages

Advantage	Explanation
Simpler Java platform	Removes a complex and rarely used security mechanism
Lower maintenance burden	JVM and JDK no longer need to maintain the Security Manager's permission machinery
Less legacy security complexity	Applications don't need to maintain complicated policy files
Clearer security architecture	Security can be enforced at OS, container, VM, and application boundaries
Better cloud-native alignment	Containers and sandboxing are generally better suited to modern deployment
Predictable behavior	Applications can no longer depend on whether a Security Manager happens to be installed
Removes obsolete permission model	Eliminates the old Java-level sandbox model
Encourages stronger isolation	Container/OS isolation can restrict the entire process rather than relying on Java permission checks

Simplifies JDK internals	Security Manager-specific implementation and privileged stack mechanisms can be removed over time
Cleaner future Java APIs	APIs no longer need to accommodate Security Manager behavior

The OpenJDK rationale is that the Security Manager had not been the primary mechanism for securing client-side Java code for many years and was rarely used for server-side security. ([OpenJDK Mail](#))

4. Disadvantages

Disadvantage	Explanation
No direct replacement	There is no one-for-one replacement for Security Manager
Legacy applications may break	Applications explicitly enabling it can fail at JVM startup
Custom security policies need redesign	java.policy-based restrictions cannot simply be carried forward
Library compatibility issues	Libraries that depend on advanced Security Manager APIs may require changes
Less fine-grained JVM-level sandboxing	Permissions are no longer enforced through the Security Manager
Migration effort	Old enterprise applications may need architectural changes
Testing changes	Tests that expect SecurityException from Security Manager checks may no longer behave the same
RMI changes	RMI remote code downloading has been removed
Security responsibility moves outward	Developers/operators must properly configure OS, containers, network policies, etc.
Future API removal	Code that continues using Security Manager APIs should eventually be migrated

5. What Happens to Existing Code?

Example 1 — Enabling it from the command line

Older Java:

```
java -Djava.security.manager -jar application.jar
```

Java 24:

Error occurred during initialization of VM

java.lang.Error:

A command line option has attempted to allow or enable
the Security Manager.

Enabling a Security Manager is not supported.

Java 24 therefore **does not merely warn**; attempting to enable it causes JVM startup to fail. ([Oracle Docs](#))

Example 2 — Installing it programmatically

Older code:

```
System.setSecurityManager(new SecurityManager());
```

Java 24:

UnsupportedOperationException:

Setting a Security Manager is not supported

([Oracle Docs](#))

Example 3 — Checking for one

```
SecurityManager sm = System.getSecurityManager();
```

```
System.out.println(sm);
```

Java 24:

null

The API remains temporarily for compatibility but its behavior has been degraded. ([Oracle Docs](#))

6. When to Use

Because this is a **removal/disablement**, "when to use" really means **when moving to Java 24 is appropriate**.

Situation	Use Java 24?	Why
New cloud-native application	✓ Yes	No reason to build around Security Manager
Containerized microservice	✓ Yes	Container/OS isolation is more appropriate
Kubernetes application	✓ Yes	Pod/container security can enforce isolation
Standard enterprise application	✓ Yes	Most applications don't require Security Manager
Spring/Jakarta application not using Security Manager	✓ Yes	Usually little migration impact
Application using java.policy	⚠ Migrate first	Policy-based security must be redesigned
Application installing custom SecurityManager	⚠ Migrate first	It will fail on Java 24
Application using advanced AccessController semantics	⚠ Migrate first	Behavior changes
Legacy plugin sandbox	⚠ Redesign	Security Manager can no longer provide the sandbox
Application dependent on RMI remote code downloading	⚠ Redesign	That mechanism was removed

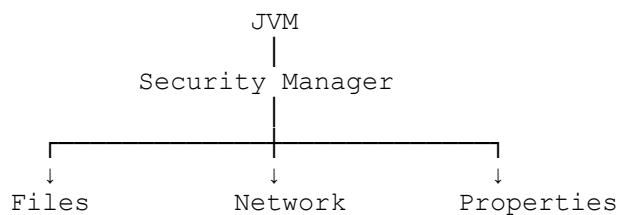
7. When NOT to Use / When to Delay Migration

Situation	Avoid/Delay Java 24?	Reason
Application explicitly enables Security Manager	✓ Delay until migrated	JVM startup will fail

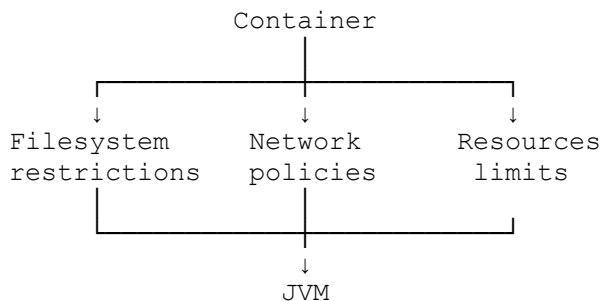
Custom SecurityManager is fundamental to architecture	✓ Delay	No direct replacement exists
Custom Policy implementation is required	✓ Delay	Security Manager policy model is disabled
Application relies on permission checks through AccessController	🔍 Investigate	Semantics change
Legacy plugin sandbox depends on Security Manager	🔍 Redesign first	Security boundary disappears
RMI remote code downloading is required	🔍 Redesign	Mechanism removed
No Security Manager dependency	✗ No reason to delay	Usually little/no impact

8. Real-Time Use Case — Containerized Microservice

Old architecture



Modern architecture



For example, instead of relying on a Java policy to prevent access to sensitive files, container/OS controls can restrict the application's filesystem and network capabilities.

Oracle recommends containers, hypervisors, and operating-system mechanisms such as Linux seccomp as alternatives for resource restrictions previously handled by the Security Manager.
([Oracle Docs](#))

9. Real-Time Use Case — Kubernetes Application

Suppose an application should not access arbitrary host resources.

Instead of:

SecurityManager
↓
Permission checks
↓
Allow/Deny

modern deployment can use:

Kubernetes
↓
Container
↓
Linux namespaces
↓
Filesystem restrictions
↓
Network policies
↓
Application

Example policy concept

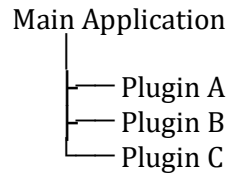
Application container

- ├── Can access /app
- ├── Can access configured temporary storage
- ├── Cannot access host filesystem
- └── Network access restricted to approved services

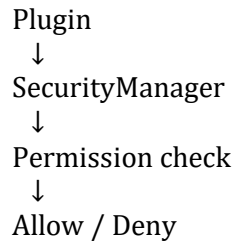
This provides isolation at a level outside the Java process.

10. Real-Time Use Case — Plugin System

Suppose an enterprise application loads third-party plugins:

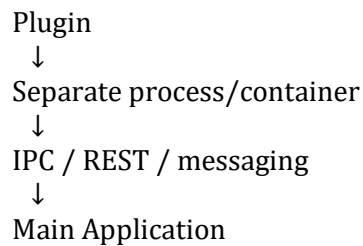


An old architecture might have attempted:



That architecture cannot be carried forward to Java 24.

A better design is:



This gives the plugin a genuine process-level security boundary.

11. Real-Time Use Case — Blocking `System.exit()`

Some applications historically used Security Manager checks to prevent untrusted code from calling:

```
System.exit(0);
```

Java 24 no longer provides the Security Manager mechanism for this.

Oracle documents an alternative using a **Java agent** that rewrites bytecode to block calls such as `System.exit(int)`. ([Oracle Docs](#))

Conceptually:

Application
↓
Java Agent
↓
Bytecode transformation
↓
System.exit() blocked/rewritten

This is useful when the requirement is **API interception**, rather than broad resource isolation.

12. Alternatives

There is **no single direct replacement** for the Security Manager. The correct alternative depends on what the old Security Manager was being used for. ([Oracle Docs](#))

Requirement previously handled by Security Manager	Recommended Alternative
Filesystem isolation	Container/OS filesystem permissions
Network isolation	Firewall / network policy / container networking
Process isolation	Containers / VM / hypervisor
CPU/memory restrictions	Container resource limits / OS controls
Sandbox untrusted application	Separate process/container/VM
Prevent dangerous API calls	Java agent / bytecode rewriting
Static restriction	Static analysis
Code transformation	Java instrumentation / Class-File API
Application authorization	Application-level authorization
Authentication	JAAS/other authentication mechanisms as appropriate
Cryptographic security	Java Cryptography Architecture APIs
Native-code isolation	OS/container isolation
JVM-level permission policy	Redesign security boundary outside Security Manager

13. Security Manager vs Modern Alternatives

Capability	Security Manager	Container	VM/Hypervisor	OS Security	Java Agent
File restriction	✓	✓	✓	✓	?
Network restriction	✓	✓	✓	✓	?
Process isolation	✗	✓	✓	✓	✗
Resource limits	Limited	✓	✓	✓	✗
API interception	Limited	✗	✗	✗	✓
Plugin isolation	?	✓	✓	✓	?
Cloud-native	✗	✓	✓	✓	?
Fine-grained Java permissions	✓	✗	✗	✗	Custom
Works on Java 24	✗	✓	✓	✓	✓

14. Security Manager vs Container Example

Requirement	Old Java approach	Modern Java 24 approach
Prevent file access	FilePermission	Container/OS filesystem restrictions
Prevent network access	SocketPermission	Network policy/firewall
Restrict process resources	Security policy	Container/OS resource limits
Isolate third-party code	Security Manager	Separate process/container
Prevent dangerous method call	Permission check	Agent/bytecode transformation where appropriate
Restrict application environment	java.policy	Container/VM/OS security

15. Impact on Enterprise Applications

Application Type	Impact
Typical Spring Boot service	Low
REST API	Low
Database application	Low
Standard web application	Low
CLI application	Low
Containerized microservice	Low
Application with custom SecurityManager	High
Application with custom Policy	High
Plugin sandbox using Security Manager	High
Legacy Java Web Start-style sandboxing	High
RMI remote code downloading	High
Application using only ordinary Java security APIs	Usually low

The OpenJDK rationale states that the change should have no impact on the vast majority of applications, libraries, and tools. ([OpenJDK Mail](#))

16. Migration Checklist

Step	Action
1	Search startup scripts for -Djava.security.manager
2	Search source code for System.setSecurityManager()
3	Search for System.getSecurityManager()
4	Search for AccessController

5	Search for custom Policy implementations
6	Search for java.policy files
7	Check libraries that depend on Security Manager
8	Identify what security requirement the manager actually provided
9	Move filesystem/network isolation to OS/container controls
10	Move application authorization into application security
11	Use agents/static analysis where API interception is genuinely required
12	Test the application on Java 24
13	Remove obsolete Security Manager configuration

Oracle also recommends using `jdeprscan` on JDK 17–23 and examining startup configuration to identify applications that depend on Security Manager APIs. ([Oracle Docs](#))

17. Advantages vs Disadvantages Summary

Category	Advantage	Disadvantage
Architecture	Simpler JVM security model	Existing sandbox architecture may need redesign
Maintenance	Less legacy JDK security code	Migration effort for legacy systems
Cloud	Fits containers/VMs well	Requires infrastructure security knowledge
Performance	Removes Security Manager machinery	No direct performance benefit should be assumed for every application
Security	Encourages stronger process/OS isolation	Java-level permission model disappears
Development	Fewer security-manager-specific APIs	Old code may stop working
Operations	Easier deployment without policy files	Infrastructure must be correctly secured

Future compatibility	Aligns with future JDK direction	Security Manager APIs will eventually disappear
-----------------------------	----------------------------------	---

18. Final Recommendation

Scenario	Recommendation
New Java 24 application	✓ Do not use Security Manager
Existing app without Security Manager	✓ Migrate normally
Containerized application	✓ Use container/OS security
Kubernetes application	✓ Use Kubernetes + OS/container controls
Untrusted plugin	✓ Prefer process/container/VM isolation
Need API interception	✓ Consider Java agents/bytecode transformation
Need application authorization	✓ Implement authorization at application/service level
Existing custom Security Manager	? Migration required before Java 24
Existing custom Policy	? Redesign security boundary
Existing java.policy configuration	? Remove/migrate
RMI remote code downloading	? Redesign

Bottom line

JEP 486 is best understood as a security architecture transition, not a security reduction feature. Java 24 permanently disables the old in-JVM Security Manager because it was rarely used effectively for modern applications and imposed significant platform complexity. There is **no single replacement**; instead, use the security boundary appropriate to the problem—**containers, VMs, OS mechanisms, network controls, application authorization, static analysis, or Java agents.** ([Oracle Docs](#))

Most important rule: don't try to recreate the old SecurityManager with another Java API. Identify *what security requirement it was solving* and move that requirement to the appropriate layer.

Java 24 — Scoped Values (Fourth Preview)

Property	Details
Java Version	24
Year	2025
Feature	Scoped Values (Fourth Preview)
JEP	JEP 487
UniqueClassName	ScopedValues
Status	Fourth Preview
Main API	java.lang.ScopedValue
Primary purpose	Safely and efficiently share immutable/contextual data with methods down the call stack without passing it through every method parameter
Especially useful with	Virtual Threads + Structured Concurrency
Alternative	ThreadLocal, method parameters, context objects, framework context mechanisms

JEP 487 is the **fourth preview** of Scoped Values in Java 24. Preview features are not permanent APIs and must be explicitly enabled. ([Oracle Docs](#))

1. What Are Scoped Values?

Normally, if a value must travel through several method calls, you pass it as a parameter:

```
handleRequest(requestId);
```

which may eventually become:

```
handleRequest(requestId)
  -> service(requestId)
    -> repository(requestId)
      -> logger(requestId);
```

This can become cumbersome when the value is **context**, rather than business data.

Scoped Values allow:

```
ScopedValue.where(REQUEST_ID, "REQ-123")
    .run() -> handleRequest();
```

Then any method called within that dynamic scope can access:

```
REQUEST_ID.get()
```

without receiving requestId as a parameter.

The JDK describes a ScopedValue as effectively an **implicit method parameter**, available only to code that has access to the ScopedValue key. ([Oracle Docs](#))

2. Basic Example

```
static final ScopedValue<String> REQUEST_ID =
    ScopedValue.newInstance();

static void handleRequest() {
    service();
}

static void service() {
    repository();
}

static void repository() {
    System.out.println(REQUEST_ID.get());
}

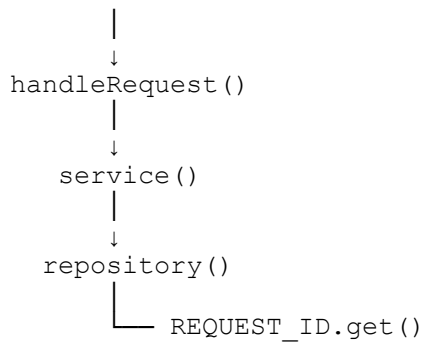
public static void main(String[] args) {
    ScopedValue.where(REQUEST_ID, "REQ-123")
        .run() -> handleRequest();
}
```

Output:

```
REQ-123
```

Conceptually:

```
main()
├── REQUEST_ID = REQ-123
```



When the run() operation completes, the binding automatically disappears. ([Oracle Docs](#))

3. Advantages and Disadvantages

Advantages	Disadvantages
Avoids passing context through many method parameters	Adds another programming model developers must understand
Safer than mutable global state	Not suitable for arbitrary mutable application state
Values are scoped to execution	A value is unavailable outside its scope
Automatically reverts after scope completion	Incorrect scope design can make code harder to understand
Designed for immutable/contextual data	Mutable objects can still cause concurrency problems
Works naturally with virtual threads	Requires preview support in Java 24
Can be inherited by structured child tasks	Unstructured thread creation requires more care
Less memory/time overhead than many ThreadLocal use cases	Too many scoped values can reduce cache effectiveness
Supports nested rebinding	Nested scopes can become confusing if overused
Good for request/transaction/security context	Not a replacement for all forms of application state
Stronger encapsulation through the	Code must have access to the key to read the

ScopedValue key	value
Integrates well with Structured Concurrency	Still a preview API in Java 24

Oracle specifically describes Scoped Values as easier to reason about than ThreadLocal, with lower space/time costs, particularly alongside virtual threads and structured concurrency. ([Oracle Docs](#))

4. Scoped Values vs Method Parameters

Aspect	Method Parameters	Scoped Values
Explicit dependency	✓ Very clear	✗ Implicit
Type safety	✓	✓
Boilerplate	Can become high	Lower for deeply propagated context
Refactoring	Often straightforward	Requires understanding scope
Business data	★★★★★	★★
Request context	★★★	★★★★★
Logging context	★★★	★★★★★
Security context	★★★	★★★★★
Hidden dependencies	Low	Higher
Best for	Business/application data	Contextual data

Important rule

Do not use Scoped Values merely to avoid writing method parameters.

Use them when the data is genuinely **contextual**, such as:

Request ID
Trace ID
Tenant context
Security identity
Locale
Transaction context

5. Scoped Values vs ThreadLocal

This is the most important comparison.

Feature	ThreadLocal	ScopedValue
Introduced	Java 1.2	Java 20+
Java 24 status	Final	Fourth Preview
Mutable value	✓ Commonly used	✗ Should generally be immutable
Lifetime	Thread-based	Lexically/dynamically scoped
Explicit cleanup	Often necessary	Automatic scope restoration
remove()	Often required	Not required for normal scope lifecycle
Rebinding	Possible but mutable	Naturally supported through nested scopes
Inheritance	InheritableThreadLocal	Structured child-thread inheritance
Virtual-thread suitability	Can have overhead at scale	Designed with modern concurrency in mind
Structured concurrency	Not designed around it	Designed to work with it
Context propagation	Possible	Natural
Risk of leaked state	Higher	Lower
Best use	Thread-associated mutable state	Immutable scoped context

The JDK documentation notes that Scoped Values are designed for structured sharing, and that their inheritance with StructuredTaskScope is efficient. ([Oracle Docs](#))

6. Why Scoped Values Are Important for Virtual Threads

Imagine a server processing:

10,000 concurrent requests



10,000 virtual threads

Each request might have:

Request ID

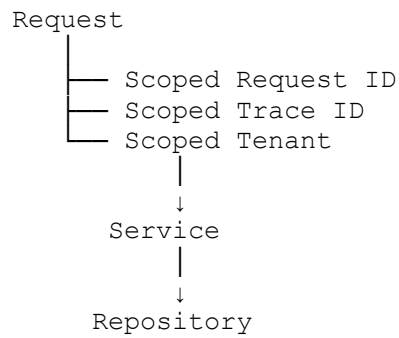
User identity

Trace ID

Tenant ID

Using traditional thread-local techniques can result in substantial per-thread context management.

Scoped Values are designed to make this type of context sharing efficient and structured.



This is one reason the JDK specifically highlights Scoped Values together with virtual threads and structured concurrency. ([Oracle Docs](#))

7. When to Use Scoped Values

Situation	Use?	Reason
Request ID propagation	✓ Excellent	Context naturally follows request execution
Distributed tracing ID	✓ Excellent	Avoids passing trace ID through every method
Tenant ID	✓ Good	Request-scoped contextual information
Security identity	✓ Good	Context can flow through call hierarchy
Locale/context	✓ Good	Useful for request-specific

configuration		immutable context
Transaction context	✓ Often	Useful when framework design supports it
Logging context	✓ Good	Correlation information can be accessed deep in call stack
Virtual-thread applications	✓ Excellent	Designed for this concurrency model
Structured-concurrency applications	✓ Excellent	Child tasks can inherit bindings
Immutable contextual data	✓ Excellent	Primary use case
Mutable shared state	✗ Usually no	Use proper synchronization/concurrency design
Application-wide configuration	✗ No	Use immutable configuration
Business entity data	✗ Usually no	Use method parameters
Long-lived state	✗ No	Scoped Values are intentionally bounded

8. When NOT to Use Scoped Values

Situation	Why Not?	Better Choice
Simple business parameter	Scoped Value hides dependency	Method parameter
Object must be modified frequently	Scoped Values are intended for contextual sharing	Explicit state
Global application configuration	Scope is unnecessary	Configuration object
Long-lived mutable state	Wrong lifecycle	Service/state object
Simple 2-3 method call	Overengineering	Method parameters

Need arbitrary thread-local mutable state	Different semantic requirement	ThreadLocal
Existing framework already manages context	Duplicate context mechanism	Framework's context
Need persistence across unrelated tasks	Scope ends with operation	Explicit storage
Need cross-process context	Scoped Values are JVM-local	HTTP/message metadata
Many dozens of scoped values	Cache effectiveness may suffer	Group related values into a record

The JDK implementation notes that applications should minimize the number of actively used scoped values because the per-thread cache is small. When many values need to be passed, grouping them into a record and binding one ScopedValue is recommended. ([Oracle Docs](#))

9. Real-Time Use Case — Request/Correlation ID

Problem

A request enters a REST service:

```

HTTP Request
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database

```

Every method shouldn't need:

```

service(requestId);
repository(requestId);
save(entity, requestId);

```

Scoped Value

```

static final ScopedValue<String> REQUEST_ID =
    ScopedValue.newInstance();

```

```

void processRequest(String requestId) {

    ScopedValue.where(REQUEST_ID, requestId)
        .run() -> controller();
}

void controller() {
    service();
}

void service() {
    repository();
}

void repository() {
    System.out.println(
        "Request = " + REQUEST_ID.get()
    );
}

```

Output:

Request = REQ-1001

Benefit

The request ID is available throughout the call tree without polluting business method signatures.

10. Real-Time Use Case — Distributed Tracing

Imagine:

```

API Gateway
↓
Order Service
↓
Payment Service
↓
Inventory Service

```

Each request has:

traceId = abc-123

Instead of:

```
processOrder(order, traceId);
```

```
chargePayment(payment, traceId);
reserveInventory(item, traceId);
```

you can establish:

```
ScopedValue.where(TRACE_ID, "abc-123")
    .run(() -> processOrder(order));
```

Deep code:

```
logger.info(
    "traceId={}",
    TRACE_ID.get()
);
```

This is an excellent contextual-data use case.

11. Real-Time Use Case — Multi-Tenant Application

Suppose one JVM serves multiple tenants:

```
Request A → Tenant A
Request B → Tenant B
Request C → Tenant C
```

You can establish:

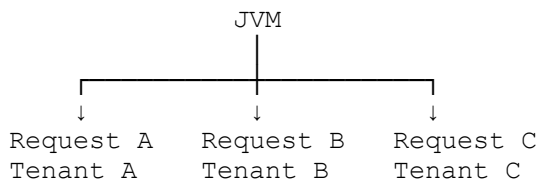
```
ScopedValue.where(TENANT_ID, "TENANT-A")
    .run(() -> processRequest());
```

Then:

```
String tenantId = TENANT_ID.get();
```

can be accessed by lower-level services.

Conceptually:



Each thread has its own binding. The API documentation explicitly states that bindings are per-thread. ([Oracle Docs](#))

12. Real-Time Use Case — Security Context

For example:

```
record UserContext(  
    String userId,  
    String role  
) {}
```

Bind it:

```
static final ScopedValue<UserContext> USER =  
    ScopedValue.newInstance();
```

```
ScopedValue.where(  
    USER,  
    new UserContext("U100", "ADMIN")  
) .run(() -> process());
```

Deep in the call stack:

```
void process() {  
    audit();  
}
```

```
void audit() {  
    UserContext user = USER.get();  
  
    System.out.println(  
        user.userId() + " / " + user.role()  
    );  
}
```

This is useful for **context propagation**, but it does **not itself perform authorization**.

For example:

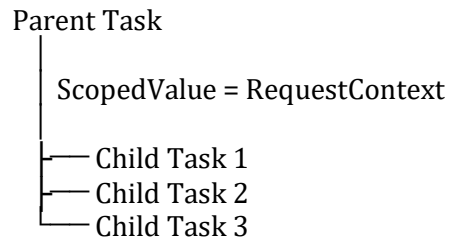
```
if (!USER.get().role().equals("ADMIN")) {  
    throw new SecurityException();  
}
```

The authorization rule is still application logic.

13. Real-Time Use Case — Structured Concurrency

Scoped Values become particularly powerful with structured concurrency.

Conceptually:



The JDK specifies that when a `StructuredTaskScope` is created, scoped-value bindings are captured and inherited by threads started with `fork()`. ([Oracle Docs](#))

For example:

```
static final ScopedValue<String> REQUEST_ID =
    ScopedValue.newInstance();

ScopedValue.where(REQUEST_ID, "REQ-123")
    .run() -> {

    // structured child tasks
    // can access REQUEST_ID
    };
```

This makes Scoped Values a natural companion to structured concurrency.

14. Rebinding / Nested Scope

Scoped Values can be rebound in a nested scope.

```
ScopedValue.where(USER, "Alice")
    .run() -> {

    System.out.println(USER.get());

    ScopedValue.where(USER, "Bob")
        .run() -> {
            System.out.println(USER.get());
        };

    System.out.println(USER.get());
    };
```

Output:

Alice
Bob
Alice

Conceptually:

```
Outer scope
USER = Alice
├── Inner scope
│   USER = Bob
└── Back to outer
    USER = Alice
```

The previous value is automatically restored after the nested scope completes. ([Oracle Docs](#))

15. Multiple Scoped Values

Java 24 supports a Carrier so several bindings can be accumulated:

```
ScopedValue.where(REQUEST_ID, "REQ-123")
    .where(TENANT_ID, "TENANT-A")
    .where(USER_ID, "U100")
    .run() -> process();
```

Conceptually:

```
Carrier
├── REQUEST_ID = REQ-123
├── TENANT_ID  = TENANT-A
└── USER_ID   = U100
```

The Carrier is immutable and thread-safe. ([Oracle Docs](#))

16. Grouping Context into a Record

If you have many related values, avoid creating dozens of separate Scoped Values.

Instead:

```
record RequestContext(
    String requestId,
```

```
String tenantId,  
String userId,  
String traceId  
) {}
```

Then:

```
static final ScopedValue<RequestContext> CONTEXT =  
    ScopedValue.newInstance();
```

Bind:

```
ScopedValue.where(  
    CONTEXT,  
    new RequestContext(  
        "REQ-1",  
        "TENANT-A",  
        "USER-10",  
        "TRACE-999"  
    )  
).run() -> process();
```

Retrieve:

```
RequestContext context = CONTEXT.get();
```

This follows the JDK's recommendation to minimize the number of scoped values and group related context when appropriate. ([Oracle Docs](#))

17. Performance Considerations

Area	Scoped Values
Context lookup	Designed to be very fast after caching
Memory	Generally efficient for scoped immutable context
Virtual threads	Very suitable
Structured concurrency	Very suitable
Thread-local cache	Uses a small per-thread cache
Too many Scoped Values	Can reduce cache hit rate
Mutable shared object	Requires synchronization

Large context object	May increase memory usage
Performance guarantee	Should still be benchmarked for your workload

The JDK implementation notes that `get()` initially searches enclosing scopes and then caches the result; excessive cycling through many scoped values can reduce cache effectiveness. ([Oracle Docs](#))

18. Scoped Value vs ThreadLocal — Practical Example

Traditional

```
static final ThreadLocal<String> REQUEST_ID =
    new ThreadLocal<>();

try {
    REQUEST_ID.set("REQ-123");

    process();

} finally {
    REQUEST_ID.remove();
}
```

The developer has to manage lifecycle carefully.

Scoped Value

```
static final ScopedValue<String> REQUEST_ID =
    ScopedValue.newInstance();

ScopedValue.where(REQUEST_ID, "REQ-123")
    .run() -> process();
```

Conceptually:

```
ThreadLocal
↓
set()
↓
execute()
↓
remove() ← developer must remember
```

```
ScopedValue
↓
```

```

where()
  ↓
execute()
  ↓
automatic restoration

```

This is one of the major reasons Scoped Values are attractive for request context.

19. Alternate Features

Requirement	Scoped Values	Alternative	When Alternative Is Better
Pass ordinary business data	✗	Method parameters	Almost always
Request context	✓	ThreadLocal	Legacy applications
Mutable thread-specific state	?	ThreadLocal	When mutation is genuinely required
Global immutable configuration	✗	static final / configuration object	Application-wide data
Multiple related context values	✓	Context/record parameter	When explicit dependencies are preferred
Async context propagation	✓	Framework context propagation	When framework already manages it
Cross-service propagation	✗	HTTP headers / messaging metadata	Context must cross JVM/process boundaries
Structured child tasks	✓	Structured Concurrency + Scoped Values	Best modern Java combination
Simple method chain	?	Method parameters	Clearer and more explicit
Reactive pipeline context	?	Reactor/other framework Context	Framework-specific reactive application
Logging context	✓	Logging MDC/context	When logging framework manages it

--	--	--	--

20. Scoped Values vs ThreadLocal vs Method Parameters

Requirement	Method Parameter	ThreadLocal	ScopedValue
Explicit dependency	★★★★★	★★	★★★
Context propagation	★★★	★★★★	★★★★★
Mutable state	★★★★★	★★★★★	★★
Immutable context	★★★★★	★★★★★	★★★★★
Automatic cleanup	★★★★★	★★	★★★★★
Virtual-thread friendly	★★★★★	★★★	★★★★★
Structured concurrency	★★★★★	★★★	★★★★★
Easy to understand	★★★★★	★★★	★★★★★
Avoid parameter pollution	★★	★★★★★	★★★★★
Best modern context mechanism	★★★	★★★	★★★★★

21. When to Choose Which?

Requirement	Best Choice
calculatePrice(product, currency)	Method parameter
processOrder(order, requestId)	Usually ScopedValue if request ID is contextual
Mutable per-thread cache	ThreadLocal
Request correlation ID	ScopedValue
Trace ID	ScopedValue

Tenant context	ScopedValue
User/security context	ScopedValue
Application configuration	Configuration object / static immutable value
Cross-service trace propagation	HTTP/message metadata + ScopedValue inside service
Reactive framework context	Framework Context
Virtual-thread request context	ScopedValue
Structured child tasks	ScopedValue + Structured Concurrency

22. Important Limitations

Limitation	Explanation
Preview API	Java 24 requires preview support
Not global state	Value exists only within its dynamic scope
Not mutable shared state	Designed primarily for immutable context
Not cross-process	Does not automatically cross service/JVM boundaries
Not a replacement for parameters	Explicit parameters remain better for business dependencies
Requires access to key	Code needs access to the ScopedValue object
Too many bindings	Can reduce cache effectiveness
Thread inheritance is structured	Not intended as unrestricted context propagation
API can change	Preview feature may be refined or removed

Java 24 preview APIs must be explicitly enabled at compile and runtime, and preview APIs can change or be removed in a future release. ([Oracle Docs](#))

23. Preview Feature Usage

Because this is a Java 24 preview feature:

```
javac --enable-preview --release 24 MyApp.java
```

Run:

```
java --enable-preview MyApp
```

Preview features are disabled by default. ([Oracle Docs](#))

24. Best-Practice Decision Table

Question	Recommendation
Is this ordinary business data?	✗ Use a method parameter
Is this request-scoped context?	✓ Scoped Value
Is the value immutable?	✓ Strong candidate
Does the value need to be modified?	⚠ Consider another design
Is it needed throughout a deep call chain?	✓ Strong candidate
Are you using virtual threads?	✓ Strong candidate
Are you using structured concurrency?	✓ Excellent candidate
Do you need automatic scope cleanup?	✓ Excellent candidate
Do you need cross-service propagation?	✗ Use protocol metadata
Do you need long-lived global state?	✗ Use another mechanism
Do you have many context values?	✓ Group them into a record
Is ThreadLocal already working and appropriate?	⚠ Migration isn't automatically necessary
Is the API going to production?	⚠ Remember Java 24 is preview

25. Overall Advantages vs Disadvantages

Category	Advantage	Disadvantage
Design	Clean context propagation	Context becomes implicit
Readability	Less parameter plumbing	Dependency is less visible
Safety	Scope automatically ends	Incorrect scope design can confuse
Concurrency	Excellent with modern concurrency	Requires structured concurrency mindset
Virtual threads	Very suitable	Preview in Java 24
Memory	Efficient for appropriate use	Too many bindings can hurt cache effectiveness
Lifecycle	Automatic restoration	Value unavailable outside scope
Maintainability	Excellent for standardized context	Overuse creates hidden dependencies
Performance	Designed for efficient access	Must benchmark application-specific workloads
Security	Scoped key acts as a capability	Security still depends on access to the key/value

Final Recommendation

Scenario	Recommendation
Request ID / correlation ID	🔗 Use Scoped Values
Distributed tracing context	🔗 Use Scoped Values
Tenant context	🔗 Use Scoped Values
User/security context	🔗 Use Scoped Values for propagation
Virtual-thread applications	🔗 Strongly consider
Structured concurrency	🔗 Excellent combination
Immutable contextual information	🔗 Ideal

Business parameters	🔗 Use method parameters
Mutable thread-specific state	🔗 Consider ThreadLocal
Global configuration	🔗 Use configuration objects/static immutable values
Cross-service propagation	🔗 Use HTTP/message metadata
Large amounts of context	🔗 Group into a context record
Java 24 production application	🔗 Be cautious because it is still preview

Bottom line

Scoped Values are essentially a modern, safer, more structured mechanism for propagating immutable context through a call hierarchy. Their strongest use case is:

Virtual Threads
 +
 Structured Concurrency
 +
 Scoped Values
 ↓
 Efficient request/context propagation

They should **not** become a general replacement for method parameters or mutable application state. Use **method parameters for business data**, **Scoped Values for contextual data**, and **ThreadLocal only when genuine thread-local mutable state is required**. ([Oracle Docs](#))

Java 24 — Primitive Types in Patterns (Second Preview)

Property	Details
Java Version	24
Year	2025
Feature	Primitive Types in Patterns, instanceof, and switch (Second Preview)
JEP	JEP 488

UniqueClassName	PrimitiveTypesPatterns
Status	Second Preview
First Preview	Java 23 — JEP 455
Main purpose	Extend pattern matching so that primitive types can participate directly in instanceof, switch, record patterns, and guarded/when patterns
Important	In Java 24, this feature was re-previewed without significant changes from Java 23. (Oracle Docs)

Note: Despite the feature name, this is broader than simply allowing int in instanceof. It extends pattern matching across **primitive type patterns, primitive record components, primitive switch selectors, and safe primitive/reference conversions.** ([Oracle Docs](#))

1. What Problem Does It Solve?

Before this feature, pattern matching was primarily designed around **reference types**.

For example:

Object value = 100;

```
if (value instanceof Integer i) {
    System.out.println(i);
}
```

But Java 24 allows primitive patterns such as:

int value = 100;

```
if (value instanceof int i) {
    System.out.println(i);
}
```

The feature also enables patterns such as:

```
switch (value) {
    case int i when i > 100 -> ...
    default -> ...
}
```

and primitive types inside record patterns.

Oracle describes the Java 24 changes as extending instanceof and switch to work with **all primitive types**. ([Oracle Docs](#))

2. Major Capabilities

Capability	What it enables	Example
Primitive type patterns	Pattern variables of primitive types	case int i
Primitive instanceof	Test primitive values using pattern syntax	value instanceof int i
Primitive switch	long, float, double, boolean selectors	switch(longValue)
Primitive record patterns	Primitive components in nested patterns	Point(int x, int y)
when clauses	Conditional primitive patterns	case int x when x > 100
Safe casting	Pattern matching can perform tested primitive conversions	if (x instanceof byte b)
Boxed/primitive interaction	Pattern matching can account for boxing/unboxing	Integer ↔ int

Java 24 extends switch selectors to all primitive types and their corresponding boxed types. ([Oracle Docs](#))

3. Advantages and Disadvantages

Advantages	Disadvantages
Makes pattern matching more uniform across primitive and reference types	Still a preview feature in Java 24
Reduces explicit casting and conversion code	Preview syntax can change in later releases
Makes complex switch logic easier to	Developers must understand primitive

express	conversion rules
Allows long, float, double, and boolean in switch	Some patterns can be less obvious than traditional code
Works naturally with record patterns	May be unnecessary for simple primitive conditions
when clauses make range-based logic readable	Complex when chains can become difficult to maintain
Can make numeric validation more declarative	Floating-point matching requires special care
Reduces repetitive if/else conversion logic	Can encourage overuse of pattern matching
Provides safer checked primitive conversions	Narrowing conversions require understanding of exactness
Makes pattern matching more orthogonal	Requires preview compiler/runtime flags
Works with both primitive and boxed values	Primitive/reference interactions can initially confuse developers
Better foundation for future pattern-matching enhancements	Not yet a permanent Java language feature

4. Simple Primitive Type Pattern

Example:

```
int value = 100;
```

```
if (value instanceof int i) {
    System.out.println("Value = " + i);
}
```

The pattern:

```
int i
```

means:

```
test value
```

```
↓
```

```
is it compatible with int?
```

↓
bind it to i

Oracle's Java 24 language documentation explicitly demonstrates primitive types being used in type patterns. ([Oracle Docs](#))

5. Primitive Types in switch

This is one of the more useful parts of the feature.

Before this feature, switch did not support all primitive types.

Java 24 preview allows:

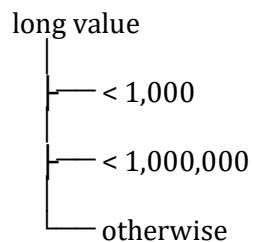
long
float
double
boolean

as selector types, in addition to the previously supported primitive types. ([Oracle Docs](#))

Example:

```
static String classify(long value) {  
    return switch (value) {  
        case long x when x < 1_000L ->  
            "Small";  
  
        case long x when x < 1_000_000L ->  
            "Medium";  
  
        default ->  
            "Large";  
    };  
}
```

Conceptually:



This can be cleaner than:

```
if (value < 1_000) {  
    ...  
} else if (value < 1_000_000) {  
    ...  
} else {  
    ...  
}
```

Oracle provides a similar example using long with when clauses. ([Oracle Docs](#))

6. double Example

```
static String rating(double rating) {  
    return switch (rating) {  
  
        case 0d ->  
            "No rating";  
  
        case double r when r > 0d && r < 2.5d ->  
            "Poor";  
  
        case double r when r >= 2.5d && r < 5d ->  
            "Good";  
  
        case 5d ->  
            "Excellent";  
  
        default ->  
            "Invalid";  
    };  
}
```

This provides a declarative way to combine:

```
type pattern  
+  
value condition  
+  
switch
```

Oracle's Java 24 documentation demonstrates this exact style of double pattern with when. ([Oracle Docs](#))

7. boolean Example

Java 24 also permits boolean as a switch selector.

Conceptually:

```
static String status(boolean active) {  
    return switch (active) {  
        case true -> "Active";  
        case false -> "Inactive";  
    };  
}
```

Although this is valid conceptually, **a normal if is usually clearer**:

```
return active ? "Active" : "Inactive";
```

This illustrates an important rule:

Just because the language allows a pattern does not mean it is the best choice.

8. Primitive Types in Record Patterns

Suppose:

```
record Point(int x, int y) {}
```

Pattern matching can directly use the primitive components:

```
static void process(Point point) {  
  
    if (point instanceof Point(int x, int y)) {  
        System.out.println(x + ", " + y);  
    }  
}
```

Conceptually:

```
Point  
├── int x  
└── int y
```

can be decomposed directly through pattern matching.

This is particularly useful when combining **records + record patterns + primitive patterns**. Java 24 includes primitive types in record-pattern contexts as part of JEP 488. ([Oracle Docs](#))

9. Safe Primitive Conversion

One of the more sophisticated capabilities is allowing pattern matching to reason about whether a primitive conversion is **exact for a particular value**.

For example, conceptually:

```
int value = 100;
```

```
if (value instanceof byte b) {  
    System.out.println("Fits in byte: " + b);  
}
```

For:

```
value = 100
```

the conversion to byte is exact.

But:

```
int value = 1000;
```

does not fit in a byte, so the pattern does not match.

The JLS defines this using **testing conversions** and whether the conversion is exact for the particular runtime value. ([Oracle Docs](#))

This is potentially useful when processing heterogeneous numeric data.

10. Why Is This Better Than Manual Casting?

Traditional code could require:

```
int value = getValue();
```

```
if (value >= Byte.MIN_VALUE &&  
    value <= Byte.MAX_VALUE) {
```

```
    byte b = (byte) value;
```

```
    process(b);  
}
```

With primitive patterns, the intent can become more declarative:

```
if (value instanceof byte b) {
    process(b);
}
```

Conceptually:

Traditional:

check range

↓

cast

↓

process

Pattern:

test + convert + bind

↓

process

However, this should be used only when it genuinely improves readability.

11. When to Use

Situation	Use?	Reason
Complex numeric classification	✓ Excellent	switch + when can express ranges clearly
long/float/double switch logic	✓ Good	Previously awkward/impossible with switch
Primitive record decomposition	✓ Good	Natural combination with record patterns
Numeric validation/conversion	✓ Good	Pattern can combine testing and binding
Heterogeneous data processing	✓ Good	Unified pattern syntax
Complex pattern-based validation	✓ Good	Declarative matching
Nested record patterns	✓ Good	Reduces extraction boilerplate
Simple if (x > 10)	✗ Usually no	Ordinary if is simpler
Simple boolean condition	✗ Usually no	if/ternary is clearer

Simple integer switch	❓ Usually unnecessary	Traditional switch already works
Performance-critical numeric loop	❓ Benchmark	Simpler primitive code may be preferable
Stable production code on Java 24	❓ Caution	Feature is preview

12. When NOT to Use

Situation	Avoid?	Better Choice
Simple comparison	✓	if
Simple boolean decision	✓	if / ternary
Simple integer switch	✓ Usually	Normal switch
Very small method	✓ Often	Conventional code
Performance-critical hot loop	❓ Benchmark first	Direct primitive operations
Team unfamiliar with preview features	❓	Stable Java constructs
Long complex when chain	❓	Extract named methods/rules
Java version must remain LTS/stable	❓	Avoid preview-dependent source
API/library intended for broad compatibility	❓	Avoid preview language syntax
Logic isn't pattern-oriented	✓	Normal control flow

13. Real-Time Use Case — Transaction Classification

Suppose transaction amounts must be classified.

```

static String classify(long amount) {
    return switch (amount) {
        case long a when a < 1_000 ->
            "SMALL";

        case long a when a < 100_000 ->
            "MEDIUM";

        case long a when a < 1_000_000 ->
            "LARGE";

        default ->
            "VERY_LARGE";
    };
}

```

Business scenario

₹0	– ₹999	→ SMALL
₹1,000	– ₹99,999	→ MEDIUM
₹100,000	– ₹999,999	→ LARGE
₹1,000,000+		→ VERY_LARGE

This is a good candidate because the classification rules are naturally represented as patterns.

14. Real-Time Use Case — Sensor Data

Suppose an IoT sensor returns a double.

```

static String classifyTemperature(double temperature) {

    return switch (temperature) {

        case double t when t < 0 ->
            "FREEZING";

        case double t when t < 20 ->
            "COLD";

        case double t when t < 35 ->
            "NORMAL";

        case double t when t < 50 ->
            "HOT";

        default ->
            "CRITICAL";
    };
}

```

```
};  
}
```

This can be useful for:

- Industrial sensors
- IoT devices
- Monitoring systems
- Environmental systems
- Server temperature monitoring

The pattern makes the classification rules visible in one place.

15. Real-Time Use Case — Financial Risk Classification

```
static String riskScore(double score) {  
  
    return switch (score) {  
  
        case double s when s < 0.20 ->  
            "LOW";  
  
        case double s when s < 0.50 ->  
            "MEDIUM";  
  
        case double s when s < 0.80 ->  
            "HIGH";  
  
        default ->  
            "CRITICAL";  
    };  
}
```

This can make a rule engine easier to read:

```
0.00  ———  0.20  ———  0.50  ———  0.80  ———  1.00  
    LOW          MEDIUM        HIGH        CRITICAL
```

16. Real-Time Use Case — Network Packet Size

Suppose packet sizes are represented by long.

```
static String packetCategory(long size) {
```

```

return switch (size) {

    case long s when s <= 512 ->
        "TINY";

    case long s when s <= 4096 ->
        "SMALL";

    case long s when s <= 65535 ->
        "NORMAL";

    default ->
        "LARGE";
};
}

```

Useful for:

- Network monitoring
- Telemetry
- Protocol processing
- Traffic classification

17. Real-Time Use Case — Record Processing

Consider:

```

record Measurement(
    int sensorId,
    double value
) {}

```

You can combine record and primitive patterns:

```

static String evaluate(Measurement measurement) {

    return switch (measurement) {

        case Measurement(int id, double value)
            when value > 100.0 ->
            "Sensor " + id + ": ALERT";

        case Measurement(int id, double value) ->
            "Sensor " + id + ": NORMAL";

    };
}

```

This combines:

Record Pattern
+
Primitive Patterns
+
when clause
+
switch

That is a much stronger use case than using primitive patterns alone.

18. Real-Time Use Case — Numeric Data Conversion

Suppose a generic numeric processing pipeline has an int and you want to use a smaller primitive type only when the value fits exactly.

Conceptually:

```
static void process(int value) {  
    if (value instanceof byte b) {  
        processByte(b);  
    } else {  
        processInt(value);  
    }  
}
```

This can be useful in:

- Serialization
- Binary protocols
- Data compression
- Storage optimization
- Protocol parsing

The JLS explicitly defines testing conversions for primitive pattern matching and uses exactness to determine whether a primitive value matches a pattern. ([Oracle Docs](#))

19. Advantages for switch

Old limitation	Java 24 preview
switch supported only certain primitive selectors	All primitive types supported

long required if/else for range logic	switch(long) possible
float/double range classification awkward	Pattern + when
boolean switch unavailable	switch(boolean)
Primitive type patterns unavailable	case int i, case long l, etc.
Primitive record components were less uniform	Primitive record patterns

Oracle confirms that Java 24 expands switch to long, float, double, and boolean, plus corresponding boxed types. ([Oracle Docs](#))

20. Performance Considerations

Aspect	Consideration
Primitive data	Avoids unnecessary object-oriented representation in the source model
Boxing	Can make primitive/reference handling more uniform
Runtime cost	Don't assume patterns are automatically faster
Numeric conversion	Exactness checks can introduce additional logic
switch	JVM can optimize switch expressions, but actual performance depends on generated code and workload
Hot loops	Benchmark before changing optimized code
Readability	Usually a bigger benefit than raw performance
Memory	Primitive patterns themselves don't imply object allocation

Important: This feature is primarily a **language expressiveness and safety improvement**, not a guaranteed performance optimization.

21. Primitive Patterns vs Traditional Alternatives

Requirement	Primitive Pattern	Traditional Alternative	Preferred
Simple numeric comparison	case int i when i > 10	if (x > 10)	If
Multiple numeric ranges	switch + when	if/else if	Primitive pattern can be clearer
Simple integer cases	switch	switch	Traditional switch
long range classification	switch(long)	if/else	Primitive pattern
double classification	switch(double)	if/else	Primitive pattern
Boolean decision	switch(boolean)	if/ternary	if/ternary
Record decomposition	Record + primitive pattern	Accessors	Pattern for complex matching
Safe numeric narrowing	Primitive pattern	Range check + cast	Primitive pattern
Performance-critical loop	Pattern	Direct arithmetic	Benchmark; often direct code
Simple conversion	Cast	Cast	Cast if safety is already guaranteed

22. Alternate Features

There isn't one single replacement for JEP 488. The best alternative depends on what you're trying to accomplish.

JEP 488 Requirement	Alternative Feature	When Alternative Is Better
Simple conditional logic	if / else	Simple conditions
Integer/char switch	Traditional switch	Existing supported selector types
Complex object matching	Pattern Matching for switch	Reference types

Object type checking	instanceof type patterns	Reference types
Record decomposition	Record Patterns	Record-based data
Numeric range checking	if / else if	Few simple ranges
Numeric conversion	Explicit cast + range check	Very simple conversion
Data classification	switch expression	When primitive pattern isn't necessary
Boolean selection	Ternary / if	Simple true/false decision
Complex validation	Dedicated validation method	Rules become too complicated

23. JEP 488 vs Pattern Matching for switch

Aspect	JEP 488	Pattern Matching for switch
Main focus	Primitive + pattern matching	Reference/object patterns
Primitive patterns	✓	Limited before JEP 488
Reference type patterns	✓ Existing ecosystem	✓
long switch	✓	Not the main purpose
double switch	✓	Not the main purpose
boolean switch	✓	Not the main purpose
Record patterns	✓ Primitive components	✓
when conditions	✓	✓
Best use	Numeric/data-pattern matching	Object hierarchy/data modeling

24. JEP 488 vs if/else

Aspect	if/else	Primitive Patterns
Simple condition	★★★★	★★★
Multiple ranges	★★★	★★★★★
Numeric classification	★★★	★★★★★
Readability for complex matching	★★★	★★★★★
Learning curve	★★★★★	★★★
Preview dependency	✗	?
Java compatibility	★★★★★	?
Pattern-oriented code	★★★	★★★★★

25. When NOT to Replace Existing Code

Don't automatically convert:

```
if (temperature > 30) {
    return "HOT";
}
```

into:

```
return switch (temperature) {
    case double t when t > 30 -> "HOT";
    default -> "NORMAL";
};
```

The second version is arguably **more complicated**.

Use primitive patterns when they make the overall decision structure clearer—not merely because the language supports them.

26. Preview Feature Considerations

Java 24 requires preview features to be explicitly enabled.

Compile:

```
javac --enable-preview --release 24 MyClass.java
```

Run:

```
java --enable-preview MyClass
```

Java's documentation states that preview features are not permanent and may change or disappear in a later Java release. ([Oracle Docs](#))

Also important for your Java-version study: **JEP 488 remained a preview feature after Java 24**; Java 25 re-previewed it as JEP 507, and Java 26 re-previewed it again as JEP 530. ([Oracle Docs](#))

27. Best-Practice Decision Table

Question	Recommendation
Do I have simple if logic?	✗ Don't use primitive patterns
Do I have several numeric ranges?	✓ Consider primitive patterns
Do I need long/float/double/boolean in switch?	✓ Strong candidate
Do I have record + numeric matching?	✓ Strong candidate
Do I need safe primitive narrowing?	✓ Consider primitive patterns
Is the code a performance-critical hot loop?	🔍 Benchmark
Is the code a public library requiring stable Java?	🔍 Avoid preview dependency
Does the team prefer simple conventional Java?	🔍 Use only where clarity improves
Is the condition easier with if?	✓ Use if
Is the matching logic complex?	✓ Consider pattern matching
Is Java 24 preview support acceptable?	✓ Then it can be used

28. Overall Comparison

Category	Primitive Types in Patterns	Traditional Java
Numeric range classification	★★★★★	★★★
long/double/float switch	★★★★★	★★
Record decomposition	★★★★★	★★★
Simple conditions	★★★	★★★★★
Code familiarity	★★★	★★★★★
Expressiveness	★★★★★	★★★
Compatibility	★★	★★★★★
Preview stability	★★	★★★★★
Complex pattern matching	★★★★★	★★★
Learning curve	★★★	★★★★★

29. Final Recommendation

Scenario	Recommendation
Simple if/else	❓ Use if/else
Simple existing switch	❓ Keep normal switch
Multiple numeric ranges	❓ Primitive patterns are useful
long/double/float classification	❓ Strong use case
boolean switch	❓ Usually if/ternary is clearer
Primitive record components	❓ Good use case
Safe primitive narrowing	❓ Good use case
Complex data classification	❓ Strong use case
Performance-critical arithmetic	❓ Benchmark

Production application requiring stable Java	⚠ Avoid preview dependency
Public library with broad Java compatibility	⚠ Avoid preview syntax

Bottom line

Primitive Types in Patterns (Second Preview) makes Java's pattern-matching model much more uniform by bringing primitive values into instanceof, switch, record patterns, and when clauses. Its biggest practical benefit is complex numeric classification and combining primitive values with record/object patterns.

It is **not a replacement for ordinary if, switch, or arithmetic operations**. For simple conditions, traditional Java remains clearer. For sophisticated data classification, especially:

```

primitive value
+
record pattern
+
when condition
+
switch

```

JEP 488 can significantly improve readability. ([Oracle Docs](#))